جامعة محمد الاول بوجدة
UNIVERSITE MOHAMMED PREMIER OUJDA
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵎⵃⵎⴷ ⴰⵎⴰⵣⵉⵖ ⵏ ⵡⵓⵊⴷⴰ

المدرسة الوطنية للذكاء الاصطناعي والرقمنة – بركان
ECOLE NATIONALE DE L'INTELLIGENCE ARTIFICIELLE ET DU DIGITAL - BERKANE
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵣⵔⴼⵜ ⵜⵉⵙⵙⵏⴰⵏⵜ ⴷ ⵜⵣⵔⴼⵜ ⵜⴰⴷⵉⵊⵉⵜⴰⵍⵜ – ⴱⵔⴽⴰⵏ

Université Mohamed Premier, Oujda, Maroc
École Nationale de l'Intelligence Artificielle et du Digital, Berkane

PROJET DE FIN D'ANNÉE

Filière : Ingénierie réseaux et sécurité informatique

Conception et Déploiement dune Plate-forme SOC/SOAR Intelligente pour lAuto-matisation de la Réponse aux Cybermenaces

RÉALISÉ PAR

AIT MBAREK Karim

IDCHAOUDI Youssef

TAHRI Mohammed

Soutenu le 01 juillet 2026 :

JURY DE SOUTENANCE

Mme. H.BENHAR	présidente du jury	ENIAD
Mme. M.BOUGHANJA	Encadrante Académique	ENIAD
Mme. Z.BAKRAOUY	Co-encadrante et Examinatrice	ENIAD

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude à **Pr. Manale Boughanja**, notre encadrante académique, pour son encadrement tout au long de ce projet. Sa disponibilité, ses conseils techniques et sa capacité à nous réorienter face aux difficultés rencontrées ont été déterminants. Son esprit critique nous a permis d'identifier nos lacunes et d'améliorer continuellement la qualité de notre travail.

Nos remerciements s'adressent également à **Mme Zineb Bakraouy**, qui nous a accompagnés lors des premières étapes de ce projet à travers plusieurs réunions de cadrage. Ses orientations initiales ont contribué à poser des bases solides pour la suite de notre démarche.

Nous remercions par ailleurs les membres du jury pour le temps qu'ils consacreront à l'évaluation de ce rapport et pour leurs remarques constructives qui ne manqueront pas d'enrichir notre réflexion.

Notre reconnaissance va aussi à l'ensemble du corps enseignant et du personnel administratif de l'**ENIAD** pour la qualité de la formation dispensée et leur accompagnement tout au long de notre cursus.

Nous tenons également à nous remercier mutuellement, en tant que membres du trinôme, pour l'esprit de collaboration, la patience et la persévérance dont chacun a fait preuve face aux obstacles techniques rencontrés au cours de ce projet.

Enfin, nous adressons nos remerciements les plus sincères à nos familles et à nos proches pour leur soutien indéfectible, leurs encouragements constants et la confiance qu'ils nous ont toujours témoignée.

Résumé

Face à la croissance exponentielle des cybermenaces et au volume croissant d'alertes de sécurité, les équipes SOC peinent à répondre efficacement sans outils d'automatisation adaptés. Ce projet propose la conception et la mise en oeuvre d'une plateforme SOC/SOAR automatisée, open source et intégrée, capable de détecter, d'enrichir, de scorer et de répondre aux incidents de sécurité en temps réel.

La solution repose sur **Wazuh** comme SIEM/XDR pour la détection, **Shuffle** comme moteur SOAR pour l'orchestration des playbooks, **TheHive** pour la gestion des cas, et une couche d'intelligence artificielle basée sur **Groq / Llama 3.3 70B** pour la détection de faux positifs et l'analyse de chaîne d'attaque. Trois scénarios de réponse automatisée ont été validés : blocage d'adresse IP sur détection de force brute, mise en quarantaine de fichier sur alerte malware, et revue analyste via un tableau de bord *human-in-the-loop* développé avec Flask.

L'ensemble de la plateforme est déployé sur un environnement de laboratoire VirtualBox en réseau isolé, garantissant la reproductibilité et la sécurité des expérimentations. Les tests réalisés confirment la cohérence du pipeline de bout en bout, depuis la détection Wazuh jusqu'à l'exécution de l'action de réponse.

Mots-clés

SOC, SOAR, SIEM, Wazuh, Shuffle, TheHive, Threat Intelligence, Intelligence Artificielle, Automatisation, Réponse aux incidents

Abstract

With the exponential growth of cyberthreats and the increasing volume of security alerts, SOC teams struggle to respond effectively without appropriate automation tools. This project presents the design and implementation of an integrated, open source SOC/SOAR platform capable of detecting, enriching, scoring and responding to security incidents in real time.

The solution relies on **Wazuh** as the SIEM/XDR for detection, **Shuffle** as the SOAR engine for playbook orchestration, **TheHive** for case management, and an artificial intelligence layer based on **Groq / Llama 3.3 70B** for false positive detection and attack chain analysis. Three automated response scenarios were validated: IP address blocking upon brute-force detection, file quarantine on malware alerts, and analyst review through a human-in-the-loop dashboard built with Flask.

The entire platform is deployed on a VirtualBox laboratory environment using an isolated host-only network, ensuring reproducibility and security of the experiments. Tests confirm the end-to-end consistency of the pipeline, from Wazuh detection to the execution of the response action.

Keywords

SOC, SOAR, SIEM, Wazuh, Shuffle, TheHive, Threat Intelligence, Artificial Intelligence, Automation, Incident Response

Sommaire

Remerciements . . . . . **i**

Résumé . . . . . **ii**

Abstract . . . . . **iii**

Liste des Abréviations . . . . . **x**

Introduction Générale **1**

1 Étude Théorique et Positionnement Technologique **4**

1.1 Centres Opérationnels de Sécurité 4

1.2 Plateformes SOAR 5

1.3 Solutions SIEM 6

1.4 Sources de Threat Intelligence 6

1.5 Intelligence Artificielle pour la Cybersécurité 7

1.6 Positionnement de Notre Solution 8

2 Conception et Architecture de la Solution **9**

2.1 Analyse des Besoins 9

2.1.1 Besoins fonctionnels 9

2.1.2 Besoins non fonctionnels 10

2.2 Architecture Globale de la Solution 10

2.2.1 Vue d'ensemble 10

2.2.2 Description des flux 11

2.3 Choix Technologiques et Justification 12

2.4 Modélisation des Playbooks 13

2.4.1 Pipeline générique 13

2.4.2 Playbook 1 Blocage IP (force brute) 13

2.4.3 Playbook 2 Quarantaine fichier (malware/FIM) 14

2.4.4 Playbook 3 Revue analyste (activité suspecte) 15

2.5 Modèle de Scoring et Logique de Triage 15

2.5.1 Principes du scoring 15

2.5.2 Critères et logique de calcul 16

2.5.3 Logique de triage et routage 17

2.5.4 Rapport d'évidence 17

3 Déploiement et Configuration de l'Infrastructure 19
3.1 Environnement de Laboratoire 19
3.1.1 Infrastructure réseau 19
3.1.2 Ports et services exposés 22
3.2 Déploiement Wazuh SIEM (VM1 192.168.56.10) 22
3.2.1 Installation du Manager 22
3.2.2 Intégration Wazuh Shuffle 23
3.2.3 Configuration de l'Active Response 23
3.3 Déploiement Shuffle SOAR (VM2 192.168.56.11) 24
3.3.1 Déploiement via Docker Swarm 24
3.3.2 Importation du workflow SOC 25
3.4 Déploiement TheHive et Score API 26
3.4.1 Déploiement TheHive 26
3.4.2 Score API Déploiement et Configuration 27
3.5 Configuration des Agents Wazuh 28
3.5.1 Agent Windows hôte void (192.168.56.1) 28

4 Implémentation et Validation des Scénarios 29
4.1 Vue d'ensemble du Pipeline d'Orchestration 29
4.2 Scénario 1 Détection Force Brute SSH et Blocage IP 32
4.2.1 Simulation de l'attaque 32
4.2.2 Enrichissement et scoring 32
4.3 Scénario 2 Détection Malware / FIM et Quarantaine 33
4.3.1 Simulation et détection FIM 33
4.3.2 Enrichissement par hachage et scoring 34
4.3.3 Mise en quarantaine du fichier 36
4.4 Scénario 3 Activité Suspecte et Revue Analyste 37
4.4.1 Simulation Création d'un compte backdoor 38
4.4.2 Analyse de chaîne d'attaque par l'IA 38
4.4.3 Webhook d'approbation Dashboard Decision 38
4.5 Couche IA AI_FP_Check et AI_Chain_Analysis 40
4.5.1 AI_Chain_Analysis Corrélation temporelle 41
4.6 Cartographie MITRE ATT&CK des Scénarios 41

5 Tableau de Bord SOC et Composants Custom 43
5.1 Conception du Tableau de Bord Flask 43
5.1.1 Objectif et cas d'usage 43
5.1.2 Arborescence du projet 43
5.2 Modèle de Données et API REST 44

5.2.1 Schéma SQLite table `alerts` . . . 44
5.2.2 Endpoints de l'API REST . . . 45
5.3 Interface Analyste . . . 46
5.3.1 Page principale liste des alertes . . . 46
5.3.2 Page de détail contexte et décision . . . 46
5.3.3 Intégration Shuffle Webhook d'approbation . . . 47
5.4 Events API Contexte Temporel pour l'IA . . . 47
5.5 Exécutables d'Active Response . . . 48
5.5.1 netsh.exe Blocage IP . . . 48
5.5.2 quarantine.exe / quarantine.ps1 . . . 48
5.5.3 isolate.exe / isolate.py . . . 49

6 Tests, Validation et Bilan . . . 51
6.1 Plan de Tests et Métriques Retenues . . . 51
6.1.1 Approche de validation . . . 51
6.1.2 Métriques retenues . . . 52
6.2 Validation Scénario 1 : Force Brute → Blocage IP . . . 52
6.2.1 Protocole de test . . . 52
6.2.2 Résultats observés . . . 53
6.2.3 Test faux positif Scénario 1 . . . 55
6.2.4 Vérification du blocage effectif . . . 55
6.3 Validation Scénario 2 : Malware/FIM → Quarantaine . . . 57
6.3.1 Protocole de test . . . 57
6.3.2 Résultats observés . . . 57
6.4 Validation Scénario 3 : Activité Suspecte → Revue Analyste . . . 59
6.4.1 Protocole de test . . . 59
6.4.2 Résultats observés . . . 59
6.4.3 Revue de l'incident et décision de l'analyste . . . 59
6.5 Comparaison MTTR Avant et Après Automatisation . . . 60
6.6 Bilan Global des Tests . . . 61
6.7 Limites de la Solution et Perspectives . . . 62
6.7.1 Limites identifiées . . . 63

Conclusion Générale . . . 65

Bibliographie . . . 68

Table des figures

2.1 Architecture globale du pipeline SOC/SOAR 12
2.2 Workflow du playbook `block_ip`. 14
2.3 Workflow du playbook `Quarantine`. 14
2.4 Workflow du playbook `analyst_review`. 15

3.1 Architecture réseau globale de la plateforme SOC/SOAR 20
3.2 Architecture technique détaillée de la plateforme SOC/SOAR 21
3.3 Tableau de bord Wazuh Manager actif et agent `void` connecté 23
3.4 Workflow SOC/SOAR complet dans l'interface Shuffle 26
3.5 Interface TheHive alertes créées automatiquement par Shuffle 27

4.1 Pipeline SOC/SOAR en cours d'exécution avec la branche `analyst_review` mise en évidence. 30
4.2 Historique des exécutions du workflow Shuffle 31
4.3 Résultat `Get_VT_HASH` dans Shuffle 34
4.4 Résultat `Get_OTX_HASH` Status SUCCESS, 0 pulses (fichier non répertorié dans OTX) 35
4.5 Réponse de la Score API noeud `Calc_Score` dans Shuffle 36
4.6 Extrait de `quarantine.ps1` localisation par SHA-256 et déplacement en quarantaine 37
4.7 Simulation S3 création du compte backdoor avec privilèges administrateur depuis PowerShell 38
4.8 S3 Ticket TheHive avec verdict IA : *Privilege Escalation*, niveau CRITICAL, 95% confiance 39
4.9 Dashboard Decision Webhook dans Shuffle routage vers Block IP, Quarantine File ou Isolate Agent 39
4.10 Cartographie MITRE ATT&CK des scénarios de la plateforme 42

5.1 Tableau de bord SOC page principale (171 alertes, 130 en attente) . . 46
5.2 Page de détail alerte `Analyst Review`, verdict IA CRITICAL 95%, événements corrélés visibles 47
5.3 `netsh.exe` lecture de l'IP source et création de la règle pare-feu Windows 48
5.4 `quarantine.ps1` localisation par SHA-256 et déplacement en quarantaine 49

5.5 `isolate.py` blocage réseau complet, exception port 1514 Wazuh maintenue 49

6.1 Notification Discord générée automatiquement après l'analyse de l'alerte et la décision de blocage de l'adresse IP malveillante. 55
6.2 Liste des règles de pare-feu Windows créées automatiquement par Wazuh Active Response pour bloquer les adresses IP identifiées comme malveillantes. 56
6.3 Exécution d'un workflow Shuffle montrant un traitement terminé en environ 27 secondes 58
6.4 Dossier `C:\Quarantine\` accès restreint confirmant l'isolation du fichier mis en quarantaine 58
6.5 Tableau de bord de revue de l'analyste présentant le résumé de l'incident, les recommandations de l'IA et les actions de réponse disponibles. . . . 60

Liste des tableaux

1.1 Niveaux d'analystes SOC 4
1.2 Comparaison des principales plateformes SOAR 5
1.3 Comparaison des solutions SIEM open source 6
1.4 Comparaison des sources de Threat Intelligence 7
1.5 Comparaison des fournisseurs LLM pour un usage SOC 7
1.6 Synthèse des outils retenus et leur rôle 8

2.1 Besoins non fonctionnels de la plateforme 10
2.2 Infrastructure de la plateforme SOC/SOAR 11
2.3 Composants technologiques de la plateforme 13
2.4 Cinq critères de scoring logique réelle de score.py 16
2.5 Correspondance score sévérité action recommandée 17

3.1 Machines virtuelles du laboratoire 20
3.2 Services et ports du laboratoire 22

4.1 Comparaison des deux noeuds IA 41

5.1 Endpoints de l'API REST du tableau de bord 45
5.2 Endpoints de la Events API 48

6.1 Métriques de validation de la plateforme 52
6.2 Détail du scoring Scénario 1 (IP 185.220.101.45) 54
6.3 Détail du scoring Scénario 2 (fichier EICAR) 57
6.4 Comparaison MTTR avant et après automatisation 61
6.5 Bilan des métriques mesurées sur les trois scénarios (5 exécutions chacun) 61
6.6 Limites de la solution et impacts 63

Liste des Abréviations

API	Application Programming Interface
AR	Active Response
CVE	Common Vulnerabilities and Exposures
DNS	Domain Name System
EDR	Endpoint Detection and Response
FIM	File Integrity Monitoring
FP	False Positive (Faux Positif)
HIDS	Host-based Intrusion Detection System
HTTP	HyperText Transfer Protocol
IoC	Indicator of Compromise
LLM	Large Language Model
MITRE	MITRE ATT&CK Framework
MTTR	Mean Time To Respond
NIST	National Institute of Standards and Technology
OTX	Open Threat Exchange (AlienVault)
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SOC	Security Operations Center
SOAR	Security Orchestration, Automation and Response
SSH	Secure Shell
TTP	Tactics, Techniques and Procedures
VM	Virtual Machine
VT	VirusTotal
XDR	Extended Detection and Response

Introduction Générale

Contexte et motivation

La transformation numérique des organisations s'accompagne d'une exposition croissante aux cybermenaces. Les attaques par force brute, les infections par malware, les élévations de privilèges non autorisées ou encore les mouvements latéraux constituent aujourd'hui des vecteurs d'intrusion courants, ciblant aussi bien les grandes entreprises que les structures de taille intermédiaire. Face à cette réalité, les équipes de sécurité opérationnelle les SOC (*Security Operations Centers*) se retrouvent confrontées à un volume d'alertes quotidiennes qui dépasse largement les capacités humaines de traitement manuel.

Dans ce contexte, l'automatisation de la détection et de la réponse aux incidents n'est plus une option mais une nécessité opérationnelle. Les plateformes SOAR (*Security Orchestration, Automation and Response*) ont émergé comme réponse à ce défi, en permettant d'orchestrer les outils de sécurité, d'automatiser les tâches répétitives et de réduire significativement le délai de réponse aux incidents. Cependant, les solutions commerciales disponibles sur le marché restent souvent inaccessibles pour les organisations à budget limité ou dans un cadre académique.

Problématique

Comment concevoir et déployer une plateforme SOC/SOAR **entièrement open source**, capable de détecter automatiquement des incidents de sécurité, d'enrichir les alertes par des sources de *threat intelligence*, de prendre des décisions de réponse assistées par intelligence artificielle, et de tracer l'ensemble du processus de la détection à l'action dans un environnement de laboratoire reproductible ?

Cette problématique soulève trois sous-questions fondamentales :

— Comment intégrer de manière cohérente des outils open source hétérogènes (SIEM, SOAR, gestion de cas, threat intelligence) au sein d'un pipeline unifié et automatisé ?

— Comment exploiter les modèles de langage de grande taille (LLM) pour réduire les faux positifs et assister l'analyste dans la qualification des incidents ?

— Comment garantir une traçabilité complète et une approche *human-in-the-loop* pour les cas ambigus, sans sacrifier la rapidité de réponse pour les menaces avérées ?

Objectifs du projet

Ce projet de fin d'année a pour objectif principal la conception et l'implémentation d'une plateforme SOC/SOAR automatisée, intégrant une couche d'intelligence artificielle, développée et validée par un trinôme d'étudiants en IRSI 2 à l'**ENIAD**. Les objectifs spécifiques sont les suivants :

1. **Déployer une infrastructure de laboratoire complète** combinant Wazuh (SIEM/XDR), Shuffle (SOAR), TheHive (gestion de cas) et un tableau de bord Flask personnalisé sur un réseau VirtualBox isolé ;
2. **Implémenter trois scénarios de réponse automatisée** couvrant les vecteurs d'attaque les plus courants : force brute SSH, détection de malware par surveillance d'intégrité de fichiers, et activité suspecte (élévation de privilèges, création de comptes) ;
3. **Intégrer une couche IA** basée sur Groq/Llama 3.3 70B pour la détection de faux positifs avant toute action automatique, et pour l'analyse de chaîne d'attaque dans les cas soumis à la revue humaine ;
4. **Enrichir les alertes** par des sources de *threat intelligence* (VirusTotal, AlienVault OTX) et calculer un score de sévérité normalisé sur 100 points, inspiré du référentiel NIST SP 800-61 ;
5. **Valider le pipeline de bout en bout** par des tests reproductibles sur chacun des trois scénarios, en mesurant le temps de traitement, la précision de classification et la cohérence des décisions IA.

Plan du rapport

Ce rapport s'articule en six chapitres, structurés de la manière suivante :

— **Chapitre 1 Étude Théorique et Positionnement Technologique** : présentation des SOC, du paradigme SOAR, des SIEM, de la *threat intelligence* et de l'IA appliquée à la cybersécurité. Chaque concept est accompagné d'une analyse

comparative des solutions disponibles, aboutissant au choix des outils retenus pour notre plateforme.

— **Chapitre 2 Conception et architecture** : analyse des besoins fonctionnels et non fonctionnels, description de l'architecture globale en cinq machines virtuelles, modélisation des trois playbooks et présentation du modèle de scoring NIST.

— **Chapitre 3 Déploiement et mise en oeuvre** : description détaillée de l'environnement de laboratoire, déploiement de chaque composant (Wazuh, Shuffle, TheHive, Score API, agents), et retour d'expérience sur les difficultés techniques rencontrées.

— **Chapitre 4 Implémentation des scénarios** : présentation des trois scénarios de détection et de réponse automatisée, avec les commandes de simulation, les flux d'exécution réels et le fonctionnement de la couche IA.

— **Chapitre 5 Tableau de bord analyste et composants custom** : conception du tableau de bord Flask, modèle de données SQLite, API REST, intégration Shuffle et présentation des exécutables d'Active Response développés sur mesure.

— **Chapitre 6 Tests, validation et bilan** : plan de tests, résultats mesurés sur les trois scénarios, bilan global des métriques et perspectives d'évolution vers un déploiement de production.

Chapitre 1

Étude Théorique et Positionnement Technologique

La multiplication des cybermenaces et la croissance du volume d'alertes de sécurité imposent aux organisations de repenser leurs outils de détection et de réponse. Ce chapitre analyse les technologies sur lesquelles repose notre plateforme et justifie les choix retenus à l'issue de chaque comparaison. Nous traitons successivement les SOC (section 1.1), le SOAR (section 1.2), les SIEM (section 1.3), la *threat intelligence* (section 1.4), l'IA appliquée à la cybersécurité (section 1.5), et le positionnement de notre solution (section 1.6).

1.1 Centres Opérationnels de Sécurité

Un **Security Operations Center** (SOC) est une entité chargée de surveiller en continu le système d'information, de détecter les incidents et d'y répondre. Il repose sur trois piliers : les *personnes*, les *processus* et les *technologies* [1].

Table 1.1 – Niveaux d'analystes SOC

Niveau	Rôle	Missions
N1	**Analyste de surveillance**	Tri des alertes, escalade, procédures standards
N2	**Analyste d'investigation**	Corrélation, qualification des incidents
N3	**Expert / Threat Hunter**	Réponse critique, amélioration des règles

La croissance du volume d'alertes génère un phénomène d'*alert fatigue* qui pousse les organisations vers l'automatisation du niveau N1 et l'assistance intelligente du niveau N2. C'est précisément pour répondre à ce défi que les plateformes SOAR ont émergé [2].

1.2 Plateformes SOAR

Le **SOAR** (*Security Orchestration, Automation and Response*) combine orchestration des outils hétérogènes, automatisation des tâches répétitives et exécution de *playbooks* [3].

Table 1.2 – Comparaison des principales plateformes SOAR

Plateforme	Modèle	Avantages	Inconvénients
Splunk SOAR	Commercial	Mature, 500+ connecteurs	Coût élevé
IBM QRadar SOAR	Commercial	Conformité, gestion de cas	Licence onéreuse
Palo Alto XSOAR	Commercial	700+ intégrations	Tarification prohibitive
Shuffle	**Open Source**	**Docker natif, API REST, léger**	**Communauté plus restreinte**
TheHive	Open Source	Gestion de cas, corrélation d'alertes	Orienté case management, pas d'orchestration

Note

Shuffle est retenu comme moteur SOAR pour son architecture Docker Swarm, sa compatibilité native avec Wazuh via webhook et la simplicité de création de playbooks visuels. **TheHive** est conservé pour son rôle de gestion des cas d'incident, complémentaire à Shuffle [4][5]..

1.3 Solutions SIEM

Un **SIEM** centralise la collecte, la normalisation et la corrélation des journaux du système d'information pour détecter les menaces et générer des alertes [6].

Table 1.3 – Comparaison des solutions SIEM open source

Solution	Avantages	Inconvénients	Modèle
Wazuh	**HIDS + XDR + Active Response, multi-OS**	**OpenSearch à configurer**	**Open Source**
Elastic SIEM	Visualisations puissantes	Ressources importantes	Freemium
Graylog	Interface intuitive	Moins de règles intégrées	Open Source

Note

Wazuh 4.14.4 est retenu pour son mécanisme d'*Active Response* permettant d'exécuter des actions directement sur les agents, et sa compatibilité native avec Shuffle [7].

1.4 Sources de Threat Intelligence

La **Threat Intelligence** contextualise les alertes brutes via la réputation des adresses IP, la classification des hachages de fichiers et les indicateurs de compromission (IoC) **mitre_attack**.

Table 1.4 – Comparaison des sources de Threat Intelligence

Source	Avantages	Inconvénients	Accès
VirusTotal	**70+ moteurs AV, IP et hash**	**Quota en version gratuite**	**API gratuite**
AlienVault OTX	**Communauté mondiale, pulses**	**Qualité variable**	**API gratuite**
MISP	Partage interne	Infrastructure à héberger	Open Source

Note

VirusTotal et **AlienVault OTX** sont utilisés en combinaison : VT pour la réputation IP et hash, OTX pour le contexte géographique et les campagnes connues **vt_apiotx_api**.

1.5 Intelligence Artificielle pour la Cybersécurité

Les **LLM** (*Large Language Models*) apportent une capacité nouvelle aux SOC : synthétiser des alertes complexes, détecter des séquences d'attaque multi-étapes et assister l'analyste [8].

Table 1.5 – Comparaison des fournisseurs LLM pour un usage SOC

Fournisseur	Avantages	Inconvénients	Modèle
Groq / Llama 3.3 70B	**Inférence rapide (LPU), API gratuite**	**Données envoyées en externe**	**Cloud gratuit**
OpenAI GPT-4o	Très performant	Payant	Cloud payant
Ollama (local)	Données locales	Ressources importantes	Open Source
Mistral API	Modèles européens, RGPD	Moins mature SOC	Cloud payant

Note

Groq / Llama 3.3 70B Versatile est retenu pour ses performances d'inférence et son API gratuite. Il intervient à deux niveaux : détection de faux positifs avant toute action automatique (`AI_FP_Check`), et analyse de chaîne d'attaque pour la revue analyste (`AI_Chain_Analysis`) [9][10]..

1.6 Positionnement de Notre Solution

Les solutions commerciales (Splunk, IBM, Palo Alto) offrent des fonctionnalités avancées mais restent inaccessibles dans un contexte académique ou de PME, tant sur le plan financier que sur le plan de la complexité de déploiement. Notre plateforme se positionne comme une **solution SOC/SOAR open source intégrée**, combinant des outils retenus à l'issue de chaque analyse comparative, et ajoutant une couche IA pour la réduction des faux positifs contribution absente des déploiements open source documentés dans la littérature.

Table 1.6 – Synthèse des outils retenus et leur rôle

Outil	Rôle dans la plateforme
Wazuh 4.14.4	SIEM/XDR détection, corrélation, Active Response
Shuffle	SOAR orchestration des playbooks et intégration APIs
TheHive	Gestion des cas et traçabilité des incidents
VirusTotal + OTX	Enrichissement par threat intelligence
Groq / Llama 3.3 70B	Couche IA détection FP et analyse de chaîne d'attaque
Flask + SQLite	Tableau de bord analyste *human-in-the-loop* [11]

Synthèse du Chapitre 1

L'analyse comparative justifie nos choix technologiques : Wazuh (SIEM), Shuffle (SOAR), VirusTotal et OTX (threat intelligence), Groq/Llama 3.3 70B (IA) et TheHive (gestion de cas). Le chapitre 2 présente la conception et l'architecture de la solution.

Chapitre 2

Conception et Architecture de la Solution

Ce chapitre présente la conception de notre plateforme SOC/SOAR. Nous commençons par l'analyse des besoins (section 2.1), puis décrivons l'architecture globale (section 2.2), justifions les choix technologiques (section 2.3), modélisons les playbooks (section 2.4) et détaillons le modèle de scoring (section 2.5).

2.1 Analyse des Besoins

2.1.1 Besoins fonctionnels

L'analyse du contexte SOC et des limites des approches manuelles a permis d'identifier les besoins fonctionnels suivants :

— **Collecte et détection** : surveiller en temps réel les endpoints Windows et Linux, détecter les attaques par force brute, les modifications de fichiers suspectes et les activités anormales via des règles de corrélation ;

— **Enrichissement automatique** : interroger des sources de threat intelligence (VirusTotal, OTX) pour contextualiser chaque alerte avec la réputation de l'adresse IP ou du hachage de fichier impliqué ;

— **Scoring et triage** : calculer un score de sévérité sur 100 points combinant plusieurs critères, et router automatiquement chaque alerte vers le playbook adapté ;

— **Réponse automatisée** : bloquer une adresse IP dans le pare-feu Windows ou mettre en quarantaine un fichier malveillant sans intervention humaine pour les cas à haute confiance ;

— **Vérification IA** : soumettre chaque décision automatique à un LLM avant exécution, afin de détecter les faux positifs et éviter des actions erronées ;

— **Revue analyste** : fournir un tableau de bord *human-in-the-loop* permettant à

l'analyste de consulter le contexte complet d'une alerte et de décider de l'action à entreprendre ;

— **Gestion des cas** : créer automatiquement un ticket dans TheHive pour chaque incident détecté, avec les enrichissements et le score associés ;

— **Notification** : informer l'équipe SOC en temps réel via Discord à chaque action automatisée ou escalade.

2.1.2 Besoins non fonctionnels

Table 2.1 – Besoins non fonctionnels de la plateforme

Critère	Description
Disponibilité	Pipeline opérationnel en continu ; chaque composant redémarre automatiquement en cas de défaillance (Docker restart policy)
Performance	Temps de traitement d'une alerte (détection action) inférieur à 30 secondes hors appel API externe
Sécurité	Réseau isolé (host-only), aucune exposition publique des composants, clés API stockées en variables d'environnement
Maintenabilité	Architecture modulaire : chaque composant est indépendant et remplaçable sans reconfiguration globale
Reproductibilité	Déployable intégralement sur VirtualBox avec un réseau host-only, sans dépendance cloud obligatoire
Traçabilité	Toute action automatique est journalisée dans les logs Wazuh, dans TheHive et dans le tableau de bord Flask

2.2 Architecture Globale de la Solution

2.2.1 Vue d'ensemble

La plateforme s'articule autour de six machines interconnectées via un réseau host-only VirtualBox (`192.168.56.0/24`) [12].. Chaque machine héberge un ou plusieurs composants du pipeline SOC/SOAR.

Table 2.2 – Infrastructure de la plateforme SOC/SOAR

Adresse IP	Rôle	Composants hébergés
`192.168.56.1`	Hôte Windows (`void`)	Agent Wazuh, scripts Active Response (`netsh.exe`, `quarantine.exe`, `isolate.exe`)
`192.168.56.10`	VM1 Wazuh Manager	Wazuh Manager/Indexer, Events API (port 8888), API Wazuh REST (port 55000)
`192.168.56.11`	VM2 Shuffle SOAR	Shuffle (Docker Swarm) : backend (5001), frontend (3001), OpenSearch
`192.168.56.12`	VM3 Services SOC	TheHive (9000), Score API (9999), Tableau de bord Flask (8080)
`192.168.56.13`	VM4 Cible SSH	Agent Wazuh Linux, serveur SSH cible pour tests force brute
`192.168.56.14`	VM5 Attaquant	Kali Linux

2.2.2 Description des flux

Le flux de traitement d'une alerte suit systématiquement les étapes suivantes :

1. **Détection** : un agent Wazuh génère une alerte lorsqu'une règle est déclenchée (force brute, modification de fichier, activité suspecte). Le Manager transmet l'alerte au webhook Shuffle via une intégration native ;
2. **Classification** : le premier noeud Shuffle (Routeur) interroge la Score API pour classifier l'alerte en l'un des quatre playbooks : `block_ip`, `isolate_agent`, `analyst_review` ou `log_only` ;
3. **Enrichissement** : selon le playbook, Shuffle interroge VirusTotal et OTX pour enrichir l'alerte avec la réputation de l'IP ou du hachage impliqué ;
4. **Scoring** : la Score API calcule un score de 0 à 100 et détermine la sévérité finale (*low*, *medium*, *high*, *critical*) ;
5. **Vérification IA** : pour les playbooks automatiques, le noeud `AI_FP_Check` (Groq/Llama 3.3 70B) valide la décision avant toute action. Si la confiance est inférieure à 70%, l'alerte est renvoyée vers le tableau de bord ;
6. **Action** : selon la décision de l'IA, Wazuh Active Response exécute le blocage IP ou la quarantaine sur l'agent cible. Pour les cas `analyst_review`, l'alerte est transmise au tableau de bord Flask ;
7. **Traçabilité** : un ticket TheHive est créé et l'équipe SOC est notifiée via Discord.

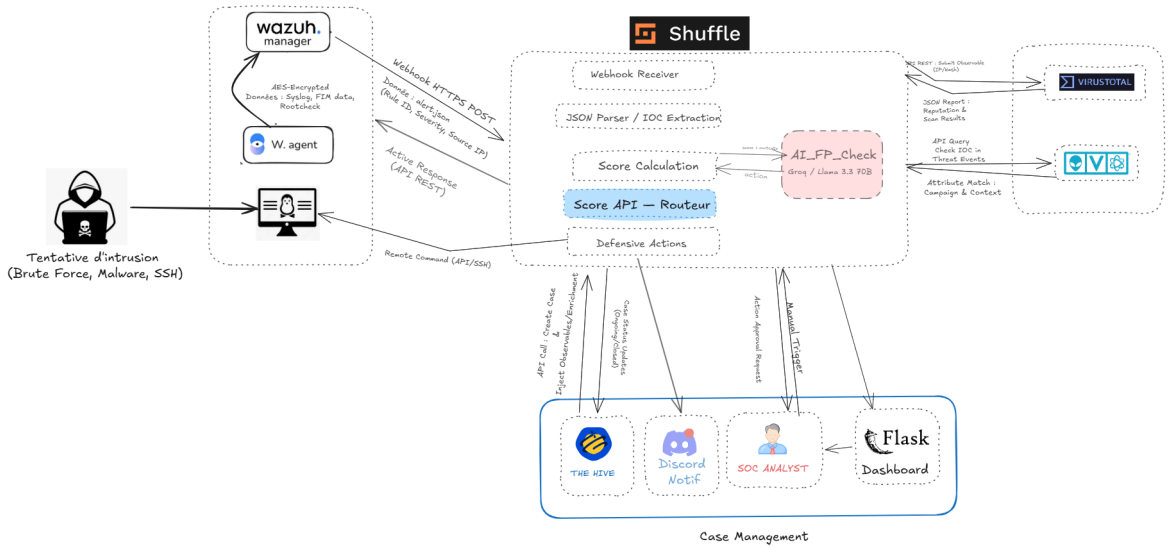


Figure 2.1 – Architecture globale du pipeline SOC/SOAR

2.3 Choix Technologiques et Justification

Les choix technologiques retenus découlent directement de l'analyse comparative réalisée au chapitre 1 (tableau 1.6). Le tableau 2.3 précise les versions déployées et le rôle exact de chaque composant dans notre pipeline.

Table 2.3 – Composants technologiques de la plateforme

Composant	Version	Rôle dans la plateforme
Wazuh Manager	4.14.4	SIEM/XDR : collecte, corrélation, détection, Active Response
Shuffle	Docker (latest)	SOAR : orchestration des playbooks, intégration des APIs, gestion du flux
TheHive	5.x	Gestion des cas et des alertes d'incidents
Score API	Python 3 / `http.server`	Classification des alertes, calcul du score NIST 0–100, routage vers playbook
VirusTotal API	v3	Enrichissement réputation IP et hachage de fichier
AlienVault OTX	v2	Threat intelligence : pulses, géolocalisation, campagnes connues
Groq / Llama 3.3 70B	API cloud	Détection de faux positifs, analyse de chaîne d'attaque
Flask Dashboard	Python 3 / SQLite	Interface analyste human-in-the-loop [11]
Discord	Webhook	Notification temps réel de l'équipe SOC
VirtualBox	7.x	Hyperviseur du laboratoire, réseau host-only

2.4 Modélisation des Playbooks

2.4.1 Pipeline générique

Tout événement Wazuh suit un pipeline commun avant de se brancher vers le playbook spécifique. Ce pipeline comprend la réception via webhook, la classification par la Score API, l'enrichissement optionnel, la vérification IA et la traçabilité finale. Le workflow complet déployé dans Shuffle est présenté au chapitre 3 (figure 3.4).

2.4.2 Playbook 1 Blocage IP (force brute)

Ce playbook est déclenché par les règles Wazuh associées aux attaques réseau et aux tentatives de force brute SSH (`5712`, `5763`, `5710`, `5716`, `40111`).

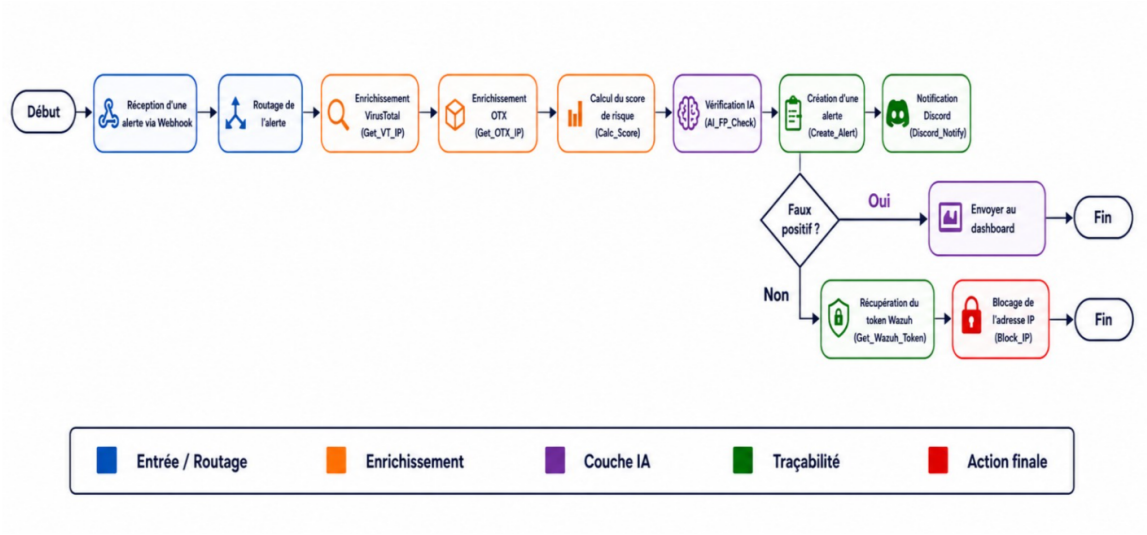


Figure 2.2 – Workflow du playbook `block_ip`.

L'action finale `Block_IP` est exécutée via l'API Wazuh Active Response, qui transmet la commande `netsh` à l'agent Windows pour bloquer l'adresse IP source dans le pare-feu.

2.4.3 Playbook 2 Quarantaine fichier (malware/FIM)

Ce playbook est déclenché par les alertes FIM et de détection de malware correspondant aux règles Wazuh `553`, `550`, `554`, `510`, `511`, `512`. L'enrichissement porte sur le hachage SHA-256 du fichier et non sur l'IP.

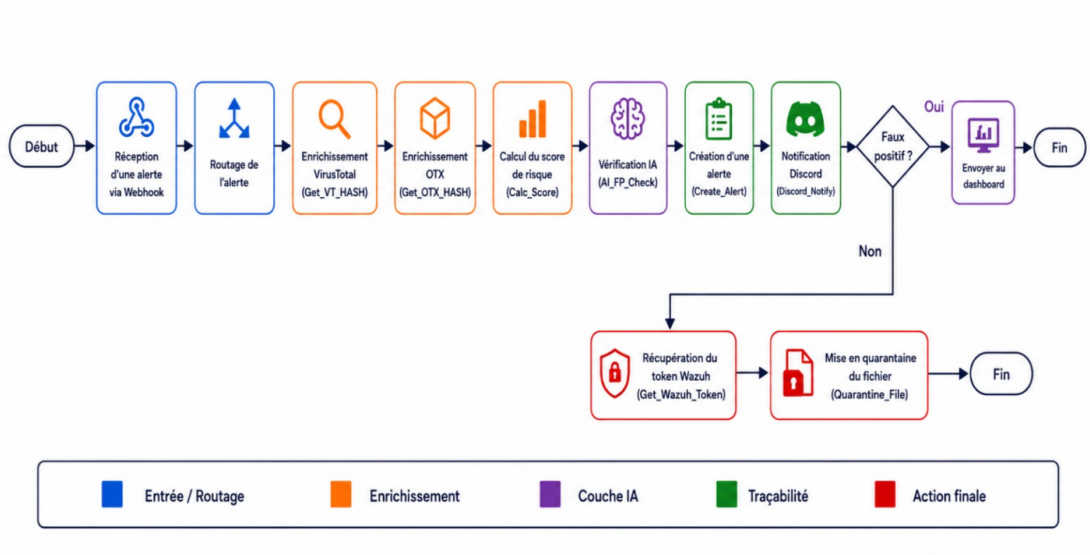


Figure 2.3 – Workflow du playbook `Quarantine`.

L'action `Quarantine_File` déclenche `quarantine.exe` sur l'agent Windows, qui localise le fichier par son SHA-256 et le déplace vers `C:\Quarantine\`.

2.4.4 Playbook 3 Revue analyste (activité suspecte)

Ce playbook couvre les activités suspectes ne devant pas faire l'objet d'une réponse automatique : création de comptes, élévation de privilèges, commandes PowerShell inhabituelles (règles `60109`, `60110`, `60106`, `5402`, `40101`, etc.).

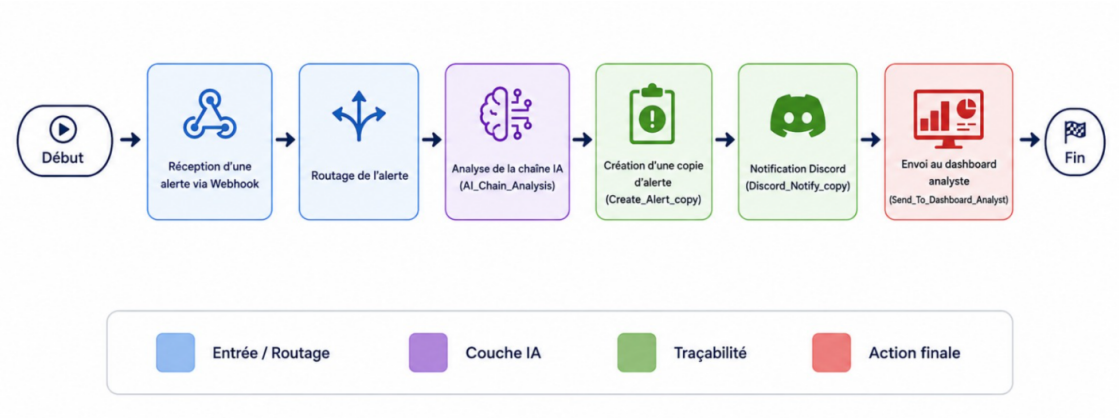


Figure 2.4 – Workflow du playbook `analyst_review`.

Attention

Ce playbook est intentionnellement semi-automatique. Le noeud `AI_Chain_Analysis` interroge l'Events API (port 8888) pour récupérer les 10 derniers événements de l'agent, les soumet à Groq/Llama 3.3 70B et génère un verdict structuré affiché sur le tableau de bord. Aucune action Wazuh n'est exécutée automatiquement.

2.5 Modèle de Scoring et Logique de Triage

2.5.1 Principes du scoring

Le moteur de scoring est implémenté dans la Score API (`score.py`) et s'inspire du référentiel **NIST SP 800-61** relatif à la gestion des incidents de sécurité [6]. Il calcule un score de sévérité entre 0 et 100 en combinant **cinq critères indépendants**, chacun évalué sur 20 points. Un bonus de 5 points est accordé lorsqu'au moins trois critères atteignent simultanément un niveau élevé, traduisant la convergence de plusieurs signaux de menace. Le score est plafonné à 100.

$$S = s_{\text{wazuh}} + s_{\text{vt}} + s_{\text{otx}} + s_{\text{mitre}} + s_{\text{freq}} + \text{bonus} \tag{2.1}$$

avec bonus $= 5$ si $|\{i \mid s_i \geq 15\}| \geq 3$, et $S = \min(S,\, 100)$.

2.5.2 Critères et logique de calcul

Table 2.4 – Cinq critères de scoring logique réelle de score.py

Critère	Score max	Logique de calcul
Wazuh Level	20 pts	Niveau ≥13 : 20 pts *(Critical)* ; ≥9 : 15 pts *(High)* ; ≥5 : 10 pts *(Medium)* ; ≥1 : 5 pts *(Low)*
VirusTotal	20 pts	Moteurs >20 : 20 pts ; ≥6 : 15 pts ; ≥1 : 8 pts ; 0 : 0 pt
OTX AlienVault	20 pts	Pulses >5 : 20 pts ; ≥3 : 15 pts ; ≥1 : 8 pts ; 0 : 0 pt
MITRE ATT&CK	20 pts	Critique (privilege, impact, exfiltration) : 20 pts ; haute (credential, execution, C2) : 15 pts ; moyenne (persistence, lateral) : 10 pts ; basse (reconnaissance, discovery) : 5 pts
Fréquence	20 pts	>20 déclenchements : 20 pts ; ≥11 : 15 pts ; ≥6 : 10 pts ; ≥3 : 5 pts
Bonus	+5 pts	Si ≥3 critères atteignent 15 pts ou plus
Total	**100 pts**	Plafonné à 100

Note

Le critère **MITRE ATT&CK** est extrait des métadonnées de l'alerte Wazuh (champ `tactic`) transmises par Shuffle. Pour le playbook `isolate_agent`, la Score API utilise les champs `vt_malicious_hash` et `otx_pulses_hash` au lieu des champs IP.

2.5.3 Logique de triage et routage

Table 2.5 – Correspondance score sévérité action recommandée

Score S	**Sévérité**	Priorité TheHive	Action recommandée
[0, 24]	**Low**	1	*Info ticket only — no action required*
[25, 49]	**Medium**	2	*Email analyst for review*
[50, 74]	**High**	3	*BLOCK IP for 1 hour + Notify analyst*
[75, 100]	**Critical**	4	*BLOCK IP for 6 hours + Urgent alert to SOC team*

Attention

Le score seul ne détermine pas le playbook : le routage primaire est effectué par `/classify` sur la base du `rule_id` Wazuh. Le score S affine uniquement la sévérité, la priorité du ticket TheHive et l'action recommandée.

2.5.4 Rapport d'évidence

À chaque calcul de score, la Score API génère automatiquement un **rapport d'évidence** (champ `evidence_report`) injecté dans la description du ticket TheHive.

```
>> SCORING BREAKDOWN
   Factor             | Score | Detail
   -------------------+-------+----------------------------
   Wazuh Level        |    15 | High (level 10)
   VirusTotal         |    15 | Malicious (18 engines)
   OTX AlienVault     |    20 | High threat (6 pulses)
   MITRE ATT&CK       |    15 | High impact (Credential)
   Frequency          |    15 | High (12 times)
   Bonus (3+ high)    |     5 |
   -------------------+-------+----------------------------
   TOTAL              |    85 | Severity: CRITICAL

>> RECOMMENDED ACTION
   BLOCK IP for 6 hours + Urgent alert to SOC team
```

Listing 2.1 – Exemple de rapport d'évidence généré par score.py

Synthèse du Chapitre 2

Ce chapitre a présenté la conception complète de notre plateforme : les besoins fonctionnels et non fonctionnels ont guidé les choix technologiques, l'architecture en six machines virtuelles couvre l'ensemble du pipeline de détection à la réponse, et les trois playbooks visuellement distincts par leur code couleur couvrent les scénarios force brute, malware et activité suspecte. Le modèle de scoring à cinq critères sur 100 points assure un triage objectif dont la logique est tracée dans le rapport d'évidence joint à chaque ticket TheHive. Le chapitre 3 détaillera le déploiement concret de chaque composant.

Chapitre 3

Déploiement et Configuration de l'Infrastructure

Ce chapitre documente la mise en place concrète de l'infrastructure de laboratoire et la configuration de chaque composant de la plateforme. L'architecture et les choix technologiques ayant été établis au chapitre 2, nous nous concentrons ici exclusivement sur les étapes de déploiement, les extraits de configuration significatifs et les obstacles techniques rencontrés. Nous présentons successivement l'environnement de laboratoire (section 3.1), le déploiement Wazuh (section 3.2), Shuffle (section 3.3), TheHive et la Score API (section 3.4), la configuration des agents (section 3.5) et les difficultés rencontrées (section **??**).

3.1 Environnement de Laboratoire

3.1.1 Infrastructure réseau

L'ensemble de la plateforme est déployé sur un hyperviseur **VirtualBox 7.x** [12] installé sur un hôte Windows 11. Les machines virtuelles communiquent via un réseau *host-only* configuré sur le sous-réseau `192.168.56.0/24`, garantissant un isolement total de l'environnement de test vis-à-vis de tout réseau externe.

Table 3.1 – Machines virtuelles du laboratoire

Adresse IP	OS	RAM	Rôle
192.168.56.1	Windows 11	(hôte natif)	Agent Wazuh `void`, endpoint supervisé, Active Response
192.168.56.10	Ubuntu 22.04	8 Go	Wazuh Manager + Indexer + Events API
192.168.56.11	Ubuntu 22.04	6 Go	Shuffle SOAR (Docker Swarm)
192.168.56.12	Ubuntu 22.04	8 Go	TheHive, Score API, Flask Dashboard
192.168.56.14	Kali Linux	4 Go	Machine attaquante

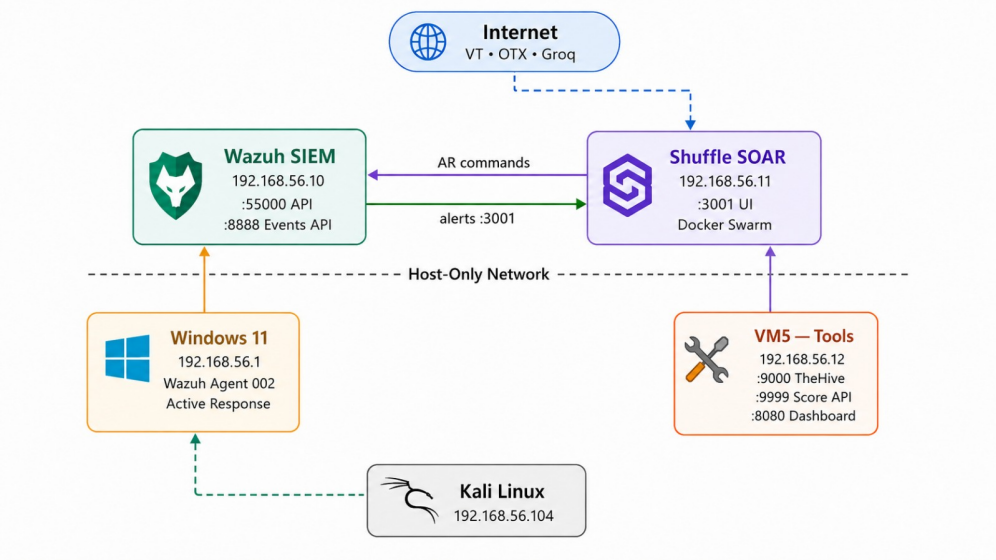


Figure 3.1 – Architecture réseau globale de la plateforme SOC/SOAR

La figure 3.1 présente une vue logique simplifiée des flux entre les composants principaux : l'agent Windows transmet ses événements au Wazuh Manager, qui relaie les alertes à Shuffle via webhook et reçoit en retour les commandes Active Response. Shuffle interroge les APIs externes (VirusTotal, OTX, Groq) pour l'enrichissement et la couche IA, tandis que la VM5 héberge TheHive, la Score API et le tableau de bord Flask. La machine Kali Linux simule les attaques sur l'agent Windows via le réseau host-only.

La figure 3.2 présente la vue technique complète de la plateforme : composants hébergés sur chaque machine, ports exposés, APIs externes et pipeline des trois scénarios.

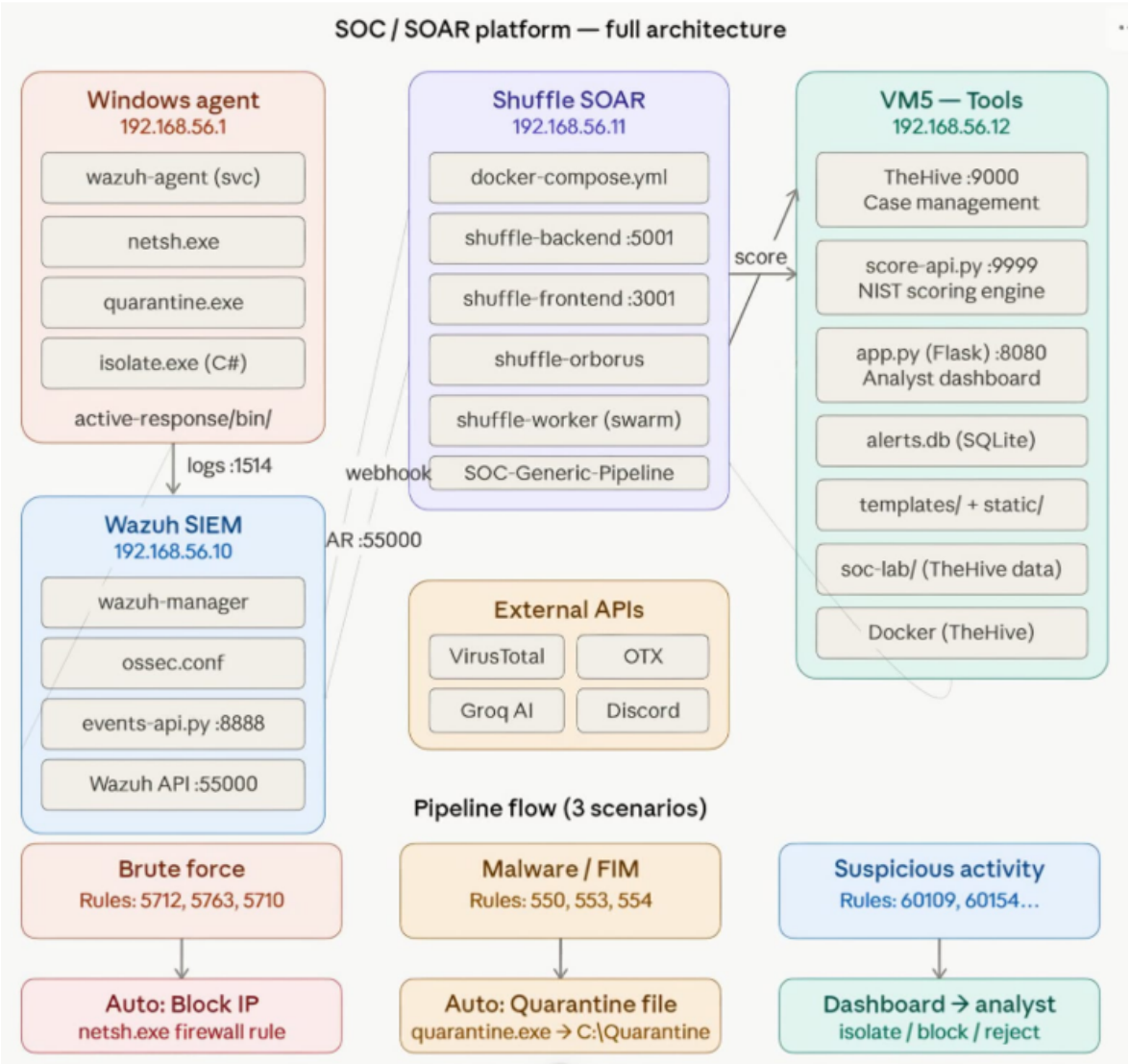


Figure 3.2 – Architecture technique détaillée de la plateforme SOC/SOAR

3.1.2 Ports et services exposés

Table 3.2 – Services et ports du laboratoire

Service	Hôte	Port	Protocole / Usage
Wazuh API REST	`192.168.56.10`	55000	HTTPS gestion agents et Active Response
Events API	`192.168.56.10`	8888	HTTP derniers événements pour `AI_Chain_Analysis`
Shuffle Backend	`192.168.56.11`	5001	HTTP réception webhooks et API SOAR
Shuffle Frontend	`192.168.56.11`	3001	HTTP interface web analyste
TheHive	`192.168.56.12`	9000	HTTP gestion des cas et alertes
Score API	`192.168.56.12`	9999	HTTP classification et scoring
Flask Dashboard	`192.168.56.12`	8080	HTTP tableau de bord analyste

3.2 Déploiement Wazuh SIEM (VM1 192.168.56.10)

3.2.1 Installation du Manager

Wazuh 4.14.4 est installé sur Ubuntu 22.04 via le script *all-in-one* officiel, qui déploie conjointement le Manager, l'Indexer OpenSearch et le Dashboard web. Une adresse IP statique est préalablement configurée sur la VM avant l'exécution du script.

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
curl -sO https://packages.wazuh.com/4.14/config.yml
# Editer config.yml : renseigner l'IP 192.168.56.10
bash wazuh-install.sh -a
```

Listing 3.1 – Installation Wazuh 4.14.4 script all-in-one

Une fois l'installation terminée, le tableau de bord web Wazuh confirme le bon fonctionnement du Manager, de l'Indexer et la connexion de l'agent 002 (figure 3.3).

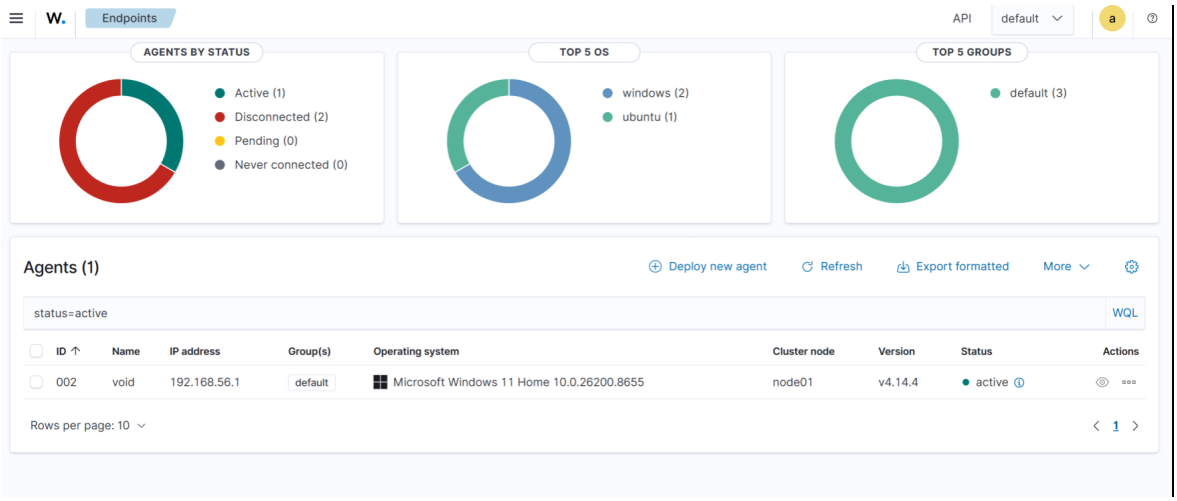


Figure 3.3 – Tableau de bord Wazuh Manager actif et agent `void` connecté

3.2.2 Intégration Wazuh Shuffle

La transmission des alertes Wazuh vers Shuffle repose sur le bloc `<integration>` du fichier `ossec.conf` du Manager. Toute alerte dont le niveau est supérieur ou égal à 5 est automatiquement envoyée au webhook Shuffle au format JSON.

```
<integration>
  <name>shuffle</name>
  <hook_url>http://192.168.56.11:5001/api/v1/hooks/WEBHOOK_ID</
     hook_url>
  <level>5</level>
  <alert_format>json</alert_format>
</integration>
```

Listing 3.2 – ossec.conf bloc d'intégration Shuffle

3.2.3 Configuration de l'Active Response

L'*Active Response* permet au Manager de déclencher à distance des actions sur les agents via l'API REST Wazuh (port 55000). Chaque commande désigne l'exécutable à invoquer sur l'agent cible. Les trois commandes utilisées dans notre plateforme sont déclarées dans `ossec.conf` :

```
<command>
  <name>netsh</name>
  <executable>netsh.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
<command>
  <name>quarantine</name>
```

```
  <executable>quarantine.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
<command>
  <name>isolate</name>
  <executable>isolate.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Listing 3.3 – ossec.conf déclaration des commandes Active Response

3.3 Déploiement Shuffle SOAR (VM2 192.168.56.11)

3.3.1 Déploiement via Docker Swarm

Shuffle est déployé sur la VM2 via **Docker Swarm** [13], gestionnaire d'orchestration natif de Docker. Ce mode garantit le redémarrage automatique des conteneurs en cas de défaillance et simplifie la gestion du cycle de vie des services. La stack Shuffle comprend quatre conteneurs : le backend (logique SOAR), le frontend (interface web), un moteur OpenSearch (indexation des logs d'exécution) et un worker Orborus (exécution des applications Shuffle).

Ce choix d'orchestration plutôt qu'un simple `docker-compose` repose sur trois critères de résilience opérationnelle, en cohérence avec le besoin non fonctionnel de disponibilité défini en section 2.1.2 (chapitre 2). Premièrement, Swarm assure le redémarrage automatique des conteneurs en cas de défaillance via sa politique de service native, sans script de supervision externe. Deuxièmement, le mode service de Swarm permettrait une montée en charge horizontale future (réplication du backend Shuffle sur plusieurs noeuds) sans modification de l'architecture, ce qu'un simple `docker-compose` ne permet pas nativement. Enfin, Swarm intègre un mécanisme de surveillance et de *rolling update*, pertinent pour un pipeline SOC devant rester opérationnel en continu. Bien que le déploiement actuel reste mono-noeud au sein du laboratoire, ce choix anticipe une extension multi-noeuds en environnement de production.

Les deux services principaux sont configurés comme suit :

```
services:
  backend:
    image: ghcr.io/shuffle/shuffle-backend:latest
    ports:
      - "5001:5001"
    environment:
```

```
      - BACKEND_HOSTNAME=192.168.56.11
      - SHUFFLE_OPENSEARCH_URL=http://opensearch:9200
    restart: always

  frontend:
    image: ghcr.io/shuffle/shuffle-frontend:latest
    ports:
      - "3001:80"
    restart: always
```

Listing 3.4 – docker-compose.yml services backend et frontend Shuffle

Note

La directive `restart: always` assure la reprise automatique des conteneurs après un redémarrage de la VM, ce qui est indispensable pour la continuité du pipeline SOC.

3.3.2 Importation du workflow SOC

Une fois Shuffle démarré, le workflow SOC est importé via l'interface graphique (*Import workflow* depuis le JSON versionné). Le noeud d'entrée **Wazuh Webhook** génère automatiquement une URL de webhook référencée dans le bloc `<integration>` du Wazuh Manager. Le noeud **Routeur** est ensuite configuré pour appeler `POST 192.168.56.12:9999/classify` afin de déterminer le playbook à exécuter pour chaque alerte entrante.

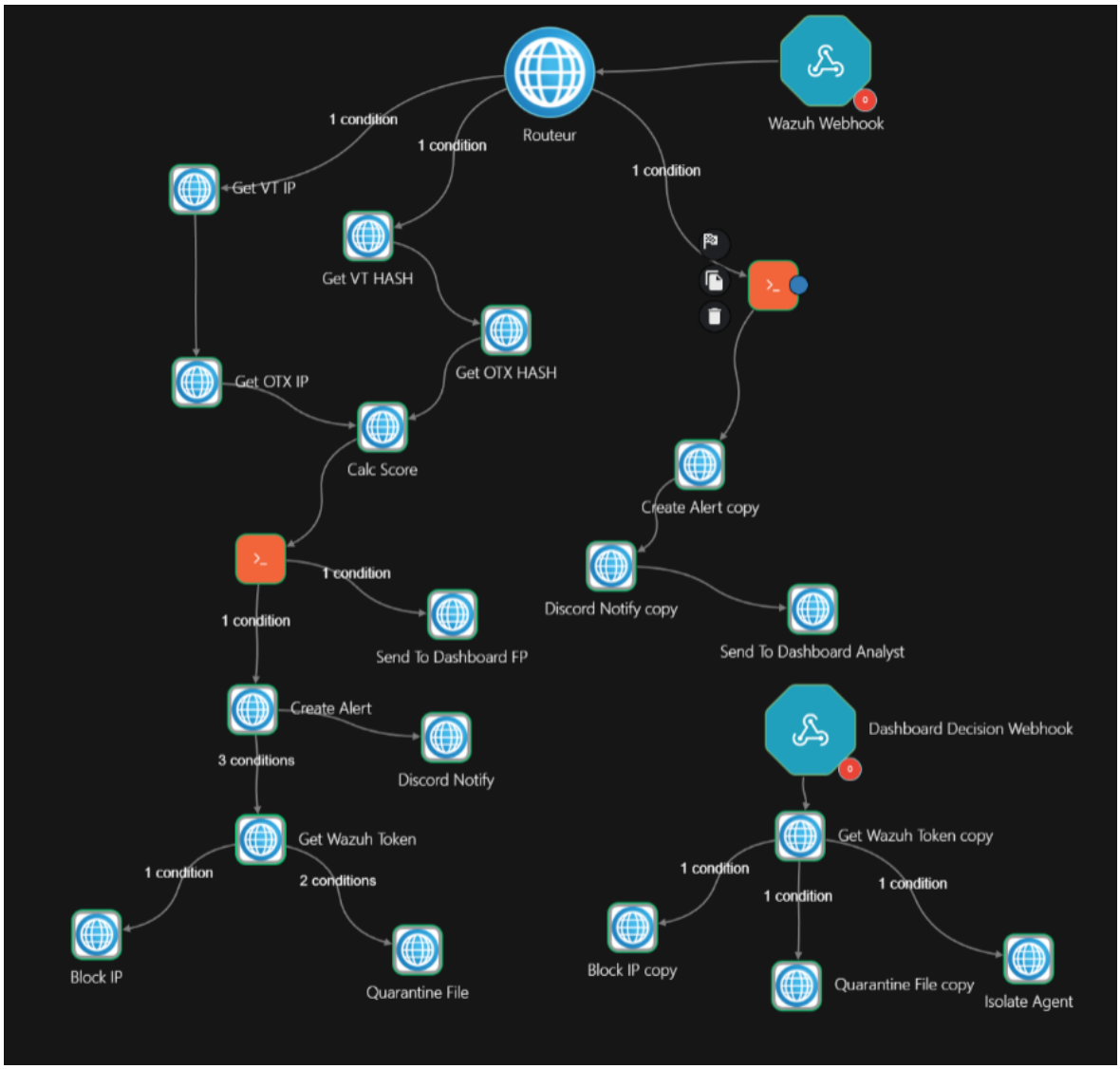


Figure 3.4 – Workflow SOC/SOAR complet dans l'interface Shuffle

Note

Le workflow a été exporté au format JSON et versionné pour garantir sa reproductibilité. Il peut être réimporté intégralement via *Import workflow* sans reconfiguration manuelle des noeuds ni des conditions.

3.4 Déploiement TheHive et Score API

3.4.1 Déploiement TheHive

TheHive 5 est déployé sur la VM3 via Docker Compose, avec Elasticsearch comme moteur de stockage. Il expose son API REST sur le port 9000, utilisée par Shuffle pour créer automatiquement un ticket pour chaque incident traité par le pipeline [5]..

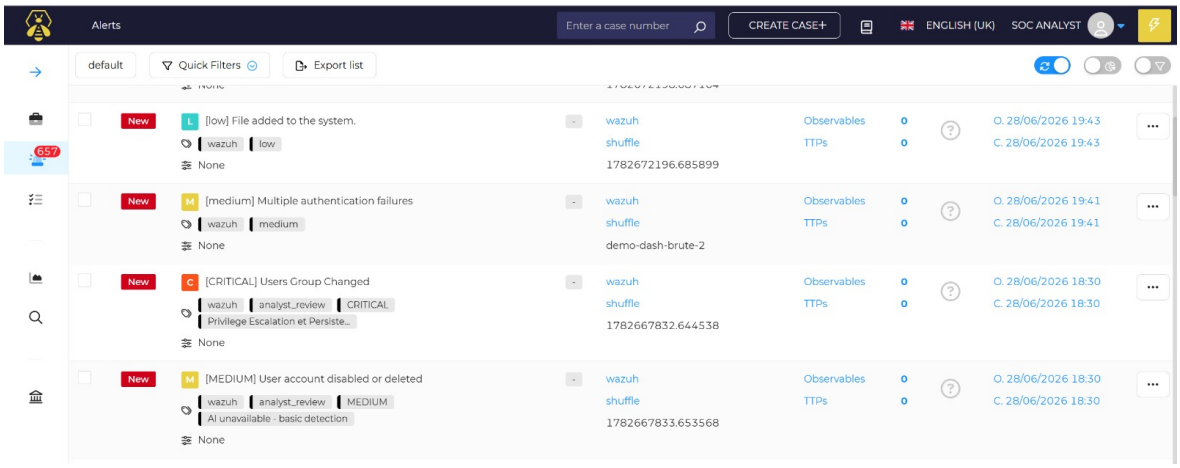


Figure 3.5 – Interface TheHive alertes créées automatiquement par Shuffle

3.4.2 Score API Déploiement et Configuration

La Score API est un service HTTP développé en Python natif (`http.server`), sans dépendance externe, déployé sur le port **9999**. Elle expose trois endpoints décrits au chapitre 2 (tableau **??**). La logique de classification et de scoring, également présentée au chapitre 2, est ici complétée par les modalités de démarrage du service :

```
bruteforce_rules = ['5712','5763','5710','5716','100203','40111']
malware_rules    = ['553','550','554','510','511','512']
suspicious_rules = [
    '60109','60110','60112',  # Comptes Windows
    '60106','92027','92066',  # PowerShell suspect
    '5402','5403',            # Sudo Linux
    '40101',                  # Nmap scan
    '92205','92213'           # Executables suspects
]

def classify_rule(rule_id):
    if   rule_id in bruteforce_rules : return "bruteforce","block_ip"
    elif rule_id in malware_rules    : return "malware","isolate_agent"
    elif rule_id in suspicious_rules : return "suspicious","analyst_review"
    else                             : return "unknown","log_only"
```

Listing 3.5 – score.py fonction `classify_rule` (routeur)

```
# Demarrage en arriere-plan
nohup python3 ~/score-api/score.py \
      > ~/score-api/score.log 2>&1 &

# Verification du service
curl http://192.168.56.12:9999/health
# {"status": "ok", "service": "score-api", "port": 9999}
```

Listing 3.6 – Démarrage et vérification de la Score API

3.5 Configuration des Agents Wazuh

3.5.1 Agent Windows hôte void (192.168.56.1)

L'agent Wazuh Windows est installé sur l'hôte physique et enregistré auprès du Manager sous l'identifiant `002`. Il assure la collecte des journaux Windows natifs (Security, System, Application) et la surveillance d'intégrité des fichiers (FIM) sur les répertoires sensibles. Les trois exécutables d'Active Response sont placés dans le répertoire dédié de l'agent :

```
C:\Program Files (x86)\ossec-agent\active-response\bin\
    netsh.exe          # Blocage IP via pare-feu Windows
    quarantine.exe     # Mise en quarantaine par SHA-256
    isolate.exe        # Isolation reseau complete de l'agent
C:\Quarantine\         # Dossier de destination (acces restreint)
```

Listing 3.7 – Arborescence Active Response agent Windows

Les trois exécutables sont compilés en C# via `csc.exe` et constituent des composants développés sur mesure. Leur fonctionnement interne est détaillé au chapitre 5.

Synthèse du Chapitre 3

Ce chapitre a documenté le déploiement complet de l'infrastructure : cinq machines VirtualBox en réseau isolé host-only, Wazuh 4.14.4 avec intégration Shuffle via webhook et déclaration des commandes Active Response, Shuffle SOAR via Docker Swarm avec importation du workflow, TheHive et la Score API sur la VM3, et l'agent Wazuh sur Windows . Le chapitre 4 présente l'implémentation et la validation des trois scénarios de détection et de réponse automatisée.

Chapitre 4

Implémentation et Validation des Scénarios

Ce chapitre présente l'implémentation concrète des trois scénarios de détection et de réponse automatisée. Pour chaque scénario, nous décrivons la simulation de l'attaque, le déroulement du pipeline Shuffle, les résultats obtenus et les preuves d'exécution. La couche IA est traitée en section 4.5.

4.1 Vue d'ensemble du Pipeline d'Orchestration

Tout événement Wazuh entre dans le pipeline par le webhook Shuffle, puis transite par le noeud **Routeur** qui interroge la Score API (`POST /classify`). La réponse oriente l'exécution vers l'une des quatre branches : `block_ip`, `Quarantine` , `analyst_review` ou `log_only`.

La figure 4.1 montre une exécution réelle du workflow avec la branche `analyst_review` mise en évidence en vert.

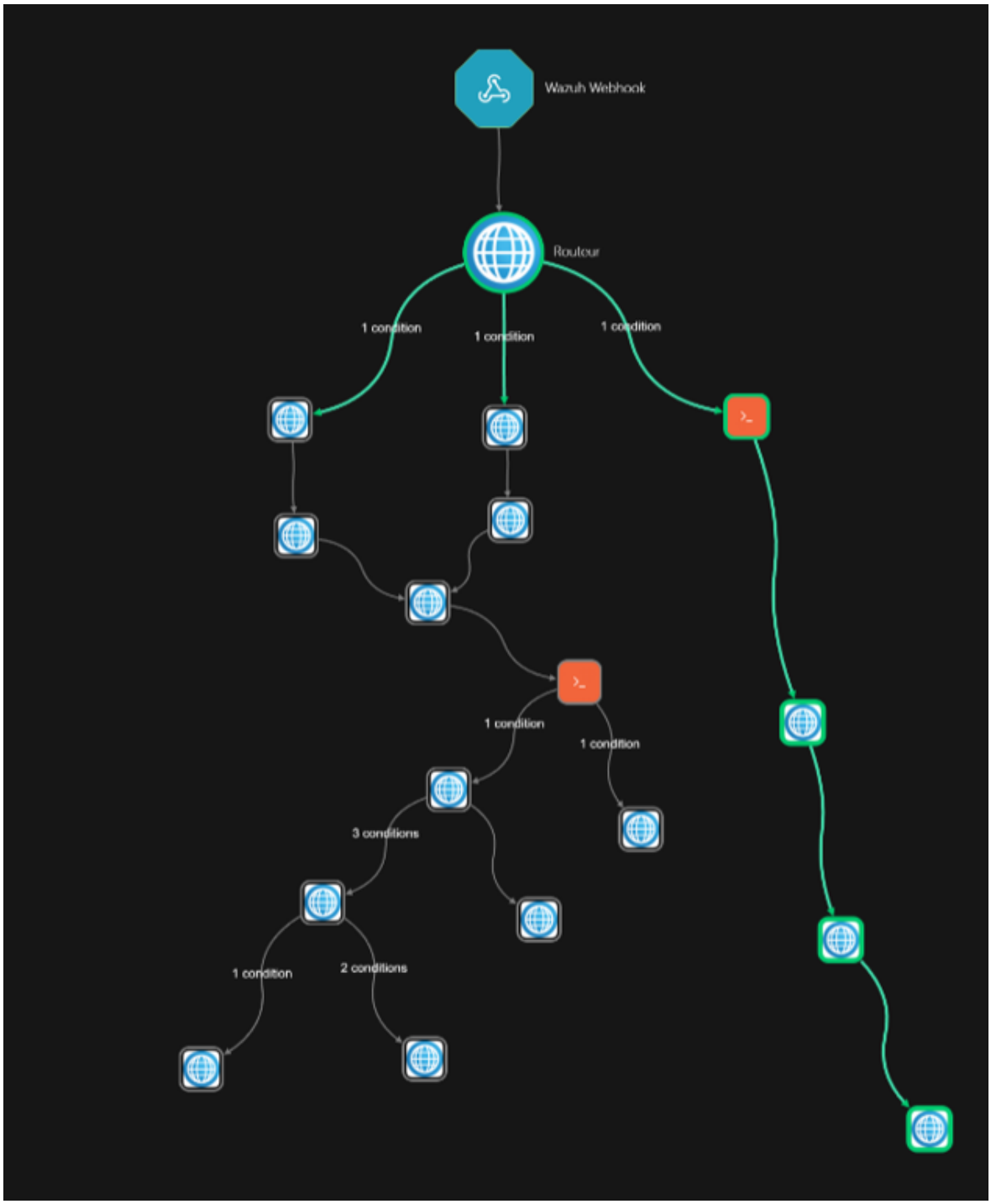


Figure 4.1 – Pipeline SOC/SOAR en cours d'exécution avec la branche `analyst_review` mise en évidence.

L'historique des exécutions Shuffle confirme le traitement continu des alertes. La figure 4.2 présente le journal des runs avec le nombre de noeuds exécutés par pipeline.

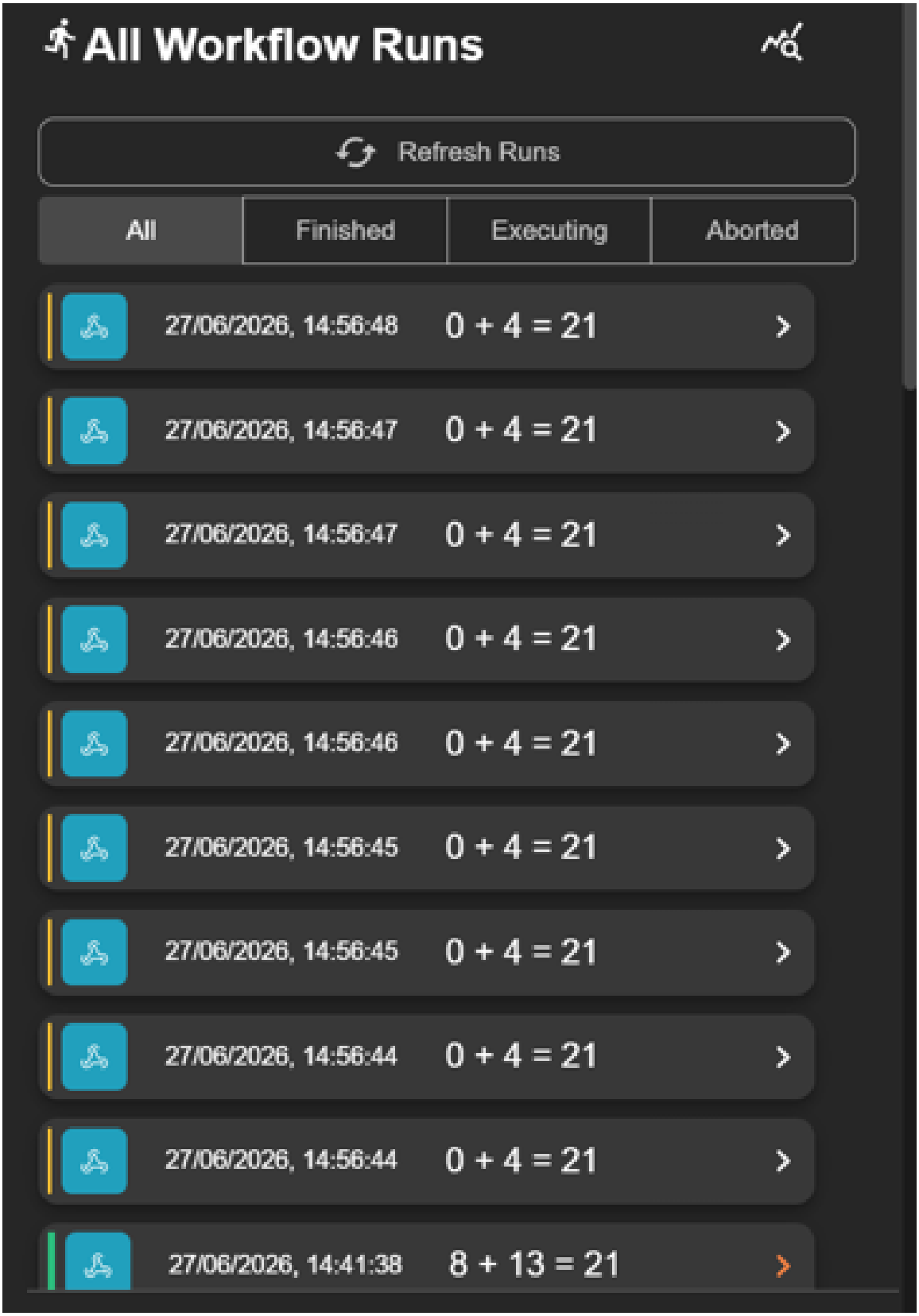


Figure 4.2 – Historique des exécutions du workflow Shuffle

4.2 Scénario 1 Détection Force Brute SSH et Blocage IP

4.2.1 Simulation de l'attaque

Afin de valider le fonctionnement de la chaîne de traitement automatisée, une alerte Wazuh simulant une attaque de type *Multiple Authentication Failures* a été injectée directement dans le workflow via le webhook Shuffle. Cette approche permet de reproduire fidèlement le comportement du SIEM tout en maîtrisant les paramètres de l'événement transmis.

L'événement contient notamment l'identifiant de la règle Wazuh **5712**, un niveau de sévérité élevé (`level = 10`), quinze déclenchements successifs (`firedtimes = 15`) ainsi que l'adresse IP source `185.220.101.45`, reconnue comme malveillante par plusieurs services de Threat Intelligence.

```
curl -X POST http://192.168.56.11:3001/api/v1/hooks/
   webhook_71aa5750-481c-4624-9c4d-8d81ce167655 \
  -H "Content-Type: application/json" \
  -d '{"severity":3,"pretext":"WAZUH Alert","title":"Multiple
     authentication failures","text":"Multiple authentication
     failures from 185.220.101.45","rule_id":"5712","timestamp
     ":"2026-06-27T10:01:00.000-0400","id":"demo-brute-002","
     all_fields":{"timestamp":"2026-06-27T10:01:00.000-0400","
     rule":{"level":10,"description":"Multiple authentication
     failures","id":"5712","firedtimes":15,"mitre":{"tactic":["
     credential-access"],"technique":["T1110"]}},"agent":{"id
     ":"002","name":"void","ip":"192.168.56.1"},"data":{"srcip
     ":"185.220.101.45"}}}'
```

Listing 4.1 – Simulation d'une alerte Wazuh via le webhook Shuffle

Après réception de cette alerte, le workflow déclenche automatiquement les différentes étapes de traitement : enrichissement de l'adresse IP par VirusTotal [14] et AlienVault OTX [15], calcul du score de risque, analyse par le module d'intelligence artificielle, puis décision d'appliquer ou non une réponse automatique selon les seuils définis.

4.2.2 Enrichissement et scoring

Shuffle interroge VirusTotal (`Get_VT_IP`) et AlienVault OTX (`Get_OTX_IP`), puis transmet les résultats au noeud `Calc_Score` via `POST 192.168.56.12:9999/`. L'action

Wazuh Active Response est ensuite déclenchée via l'API REST du Manager :

```
PUT https://192.168.56.10:55000/active-response?agents_list=002
{
  "command": "!netsh",
  "arguments": ["srcip"],
  "alert": {
    "data": { "srcip": "192.168.56.14" }
  }
}
```

Listing 4.2 – Requête Active Response blocage IP

L'exécutable `netsh.exe` crée une règle de pare-feu Windows bloquant toute connexion entrante depuis l'IP source. Son fonctionnement interne est le suivant :

```
# Lit l'IP source depuis le JSON Wazuh (stdin)
# Cree la regle de blocage :
netsh advfirewall firewall add rule ^
  name="WAZUH ACTIVE RESPONSE BLOCKED IP" ^
  remoteip=<srcip> action=block
```

Listing 4.3 – Comportement de `netsh.exe` règle créée

4.3 Scénario 2 Détection Malware / FIM et Quarantaine

4.3.1 Simulation et détection FIM

Le scénario est déclenché en déposant le fichier de test EICAR sur l'hôte Windows, reconnu par tous les moteurs antivirus :

```
Set-Content -Path "$env:USERPROFILE\Downloads\eicar.txt" `
  -Value 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-
     TEST-FILE!$H+H*'
```

Listing 4.4 – Création du fichier EICAR test FIM

Wazuh FIM détecte la création du fichier en moins de 5 secondes (règle **554** *File added to the system*) et transmet l'alerte avec le hachage SHA-256 du fichier.

4.3.2 Enrichissement par hachage et scoring

Contrairement au scénario 1, l'enrichissement porte sur le **hachage SHA-256**. Les noeuds `Get_VT_HASH` et `Get_OTX_HASH` interrogent les APIs respectives. La figure 4.3 présente la réponse VirusTotal pour le hachage du fichier, et la figure 4.4 celle d'AlienVault OTX.

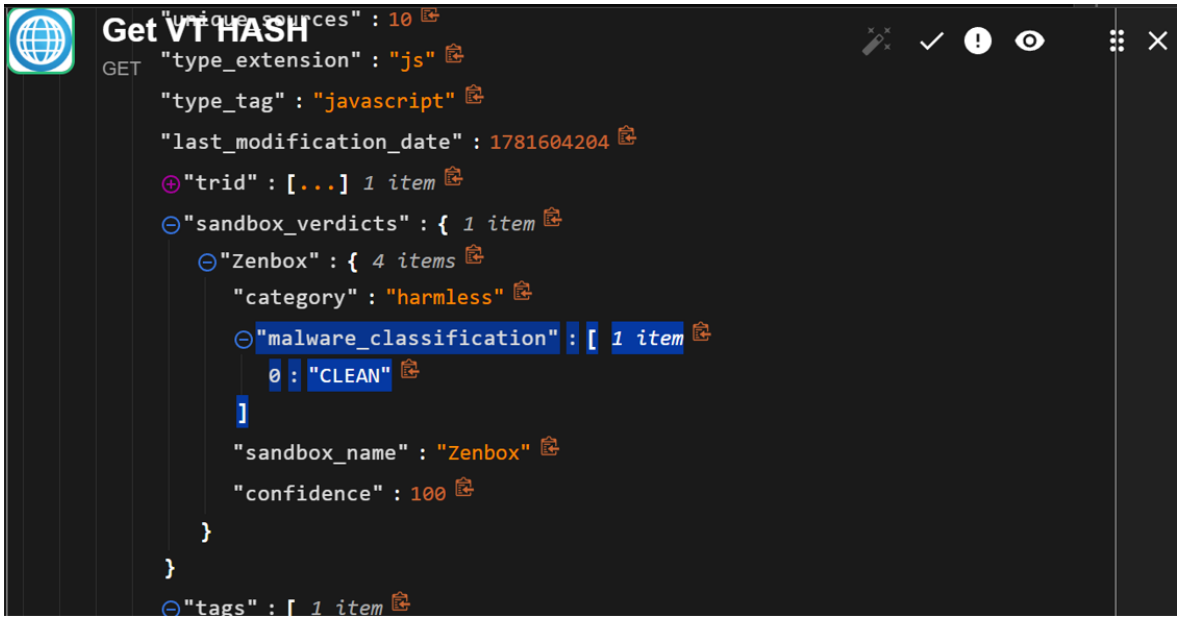


Figure 4.3 – Résultat `Get_VT_HASH` dans Shuffle

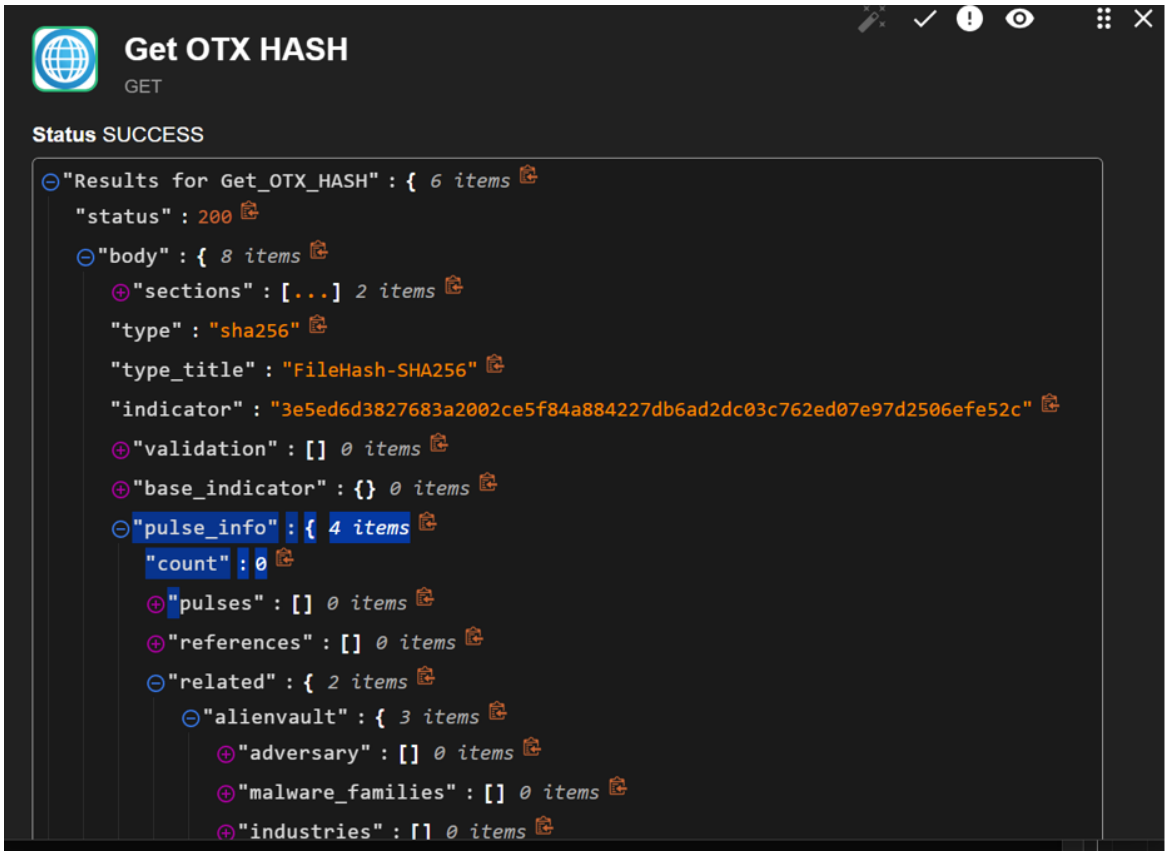


Figure 4.4 – Résultat `Get_OTX_HASH` Status SUCCESS, 0 pulses (fichier non répertorié dans OTX)

Les résultats sont transmis au noeud `Calc_Score`. La figure 4.5 montre la réponse JSON de la Score API : score 35/100, sévérité *medium*, action recommandée *Email analyst for review*.

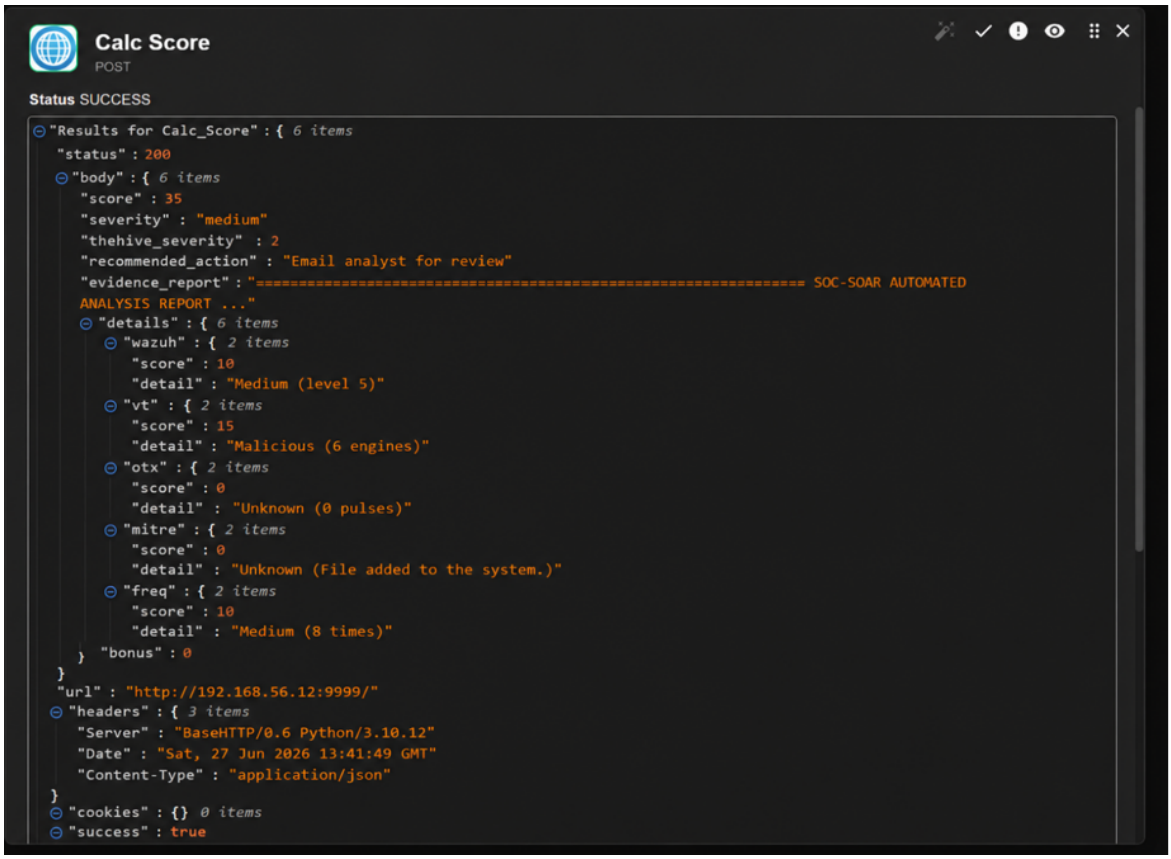


Figure 4.5 – Réponse de la Score API noeud `Calc_Score` dans Shuffle

4.3.3 Mise en quarantaine du fichier

Après validation par `AI_FP_Check`, Shuffle déclenche l'Active Response Wazuh :

```
PUT https://192.168.56.10:55000/active-response?agents_list=002
{
  "command": "!quarantine",
  "alert": {
    "syscheck": {
      "sha256_after": "3e5ed6d3827683a2002ce5f84a884227..."
    }
  }
}
```

Listing 4.5 – Requête Active Response quarantaine fichier

L'exécutable `quarantine.exe` appelle le script `quarantine.ps1` qui localise le fichier par son SHA-256 et le déplace vers `C:\Quarantine\`. La figure 4.6 illustre la logique du script.

`quarantine.ps1` — the important part:

powershell

```powershell
$input = [Console]::In.ReadToEnd()
if ($input -match '"sha256"\s*:\s*"([a-f0-9]+)"') {
    $sha256 = $matches[1]
}
Get-ChildItem -Path "C:\Users\void\Downloads" -Recurse -File | ForEach-Object {
    $hash = (Get-FileHash $_.FullName -Algorithm SHA256).Hash
    if ($hash -eq $sha256) {
        Move-Item $_.FullName $quarantineDir -Force
    }
}
```

Reads SHA256 from Wazuh → finds file by hash → moves to quarantine.

Figure 4.6 – Extrait de `quarantine.ps1` localisation par SHA-256 et déplacement en quarantaine

Le journal `active-responses.log` de l'agent Windows confirme l'exécution effective de la mise en quarantaine :

```
2026/06/29 23:47:29 active-response/bin/quarantine: Started
2026/06/29 23:47:29 active-response/bin/quarantine: Searching
    by SHA256: 3
       e5ed6d3827683a2002ce5f84a884227db6ad2dc03c762ed07e
    97d2506efe52c
2026/06/29 23:47:29 active-response/bin/quarantine: Found
    file by hash: C:\Users\void\Downloads\eicar.com
2026/06/29 23:47:29 active-response/bin/quarantine:
    QUARANTINED: C:\Users\void\Downloads\eicar.com ->
    C:\Quarantine\eicar.com
2026/06/29 23:47:29 active-response/bin/quarantine: Ended
```

Listing 4.6 – Extrait de `active-responses.log` exécution de `quarantine.exe` sur l'agent `void`

4.4 Scénario 3 Activité Suspecte et Revue Analyste

4.4.1 Simulation Création d'un compte backdoor

La simulation consiste à créer un compte utilisateur avec privilèges administrateur sur l'hôte Windows, action caractéristique d'une élévation de privilèges post-compromission :

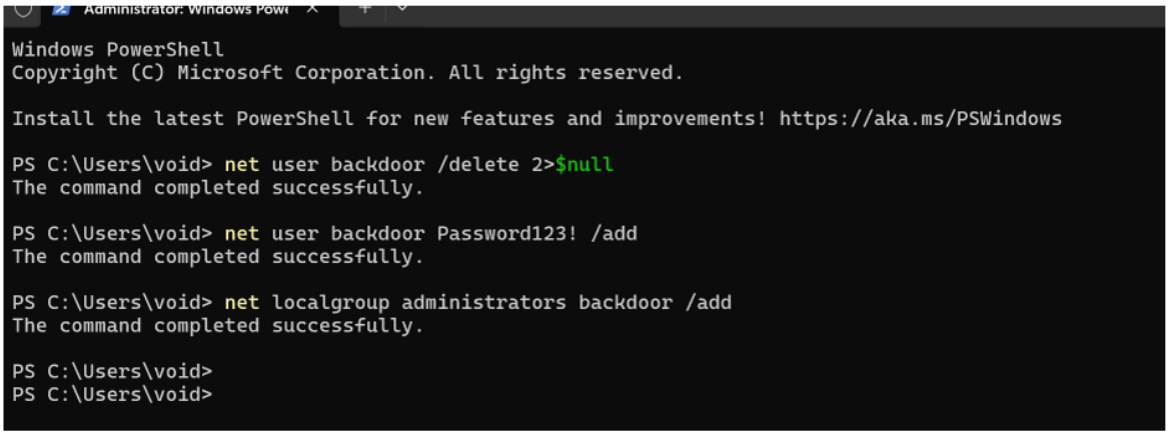


Figure 4.7 – Simulation S3 création du compte backdoor avec privilèges administrateur depuis PowerShell

Ces commandes déclenchent les règles Wazuh **60109** (*User account enabled or created*) et **60112** (*User added to group*). La Score API classe l'alerte en `analyst_review` : aucune action automatique n'est exécutée.

4.4.2 Analyse de chaîne d'attaque par l'IA

Le noeud `AI_Chain_Analysis` interroge l'Events API, récupère les 10 derniers événements de l'agent et soumet la chronologie à Groq/Llama 3.3 70B. Le verdict produit est injecté dans le ticket TheHive et transmis au tableau de bord.

La figure 4.8 montre le ticket TheHive créé automatiquement pour ce scénario, avec le verdict IA intégré dans la description.

4.4.3 Webhook d'approbation Dashboard Decision

Une fois la décision de l'analyste prise via le tableau de bord, elle est transmise à Shuffle via le **Dashboard Decision Webhook**. Ce second webhook route l'exécution selon la valeur du champ `decision` : blocage IP, quarantaine ou rejet. La figure 4.9 illustre cette branche d'approbation dans Shuffle.

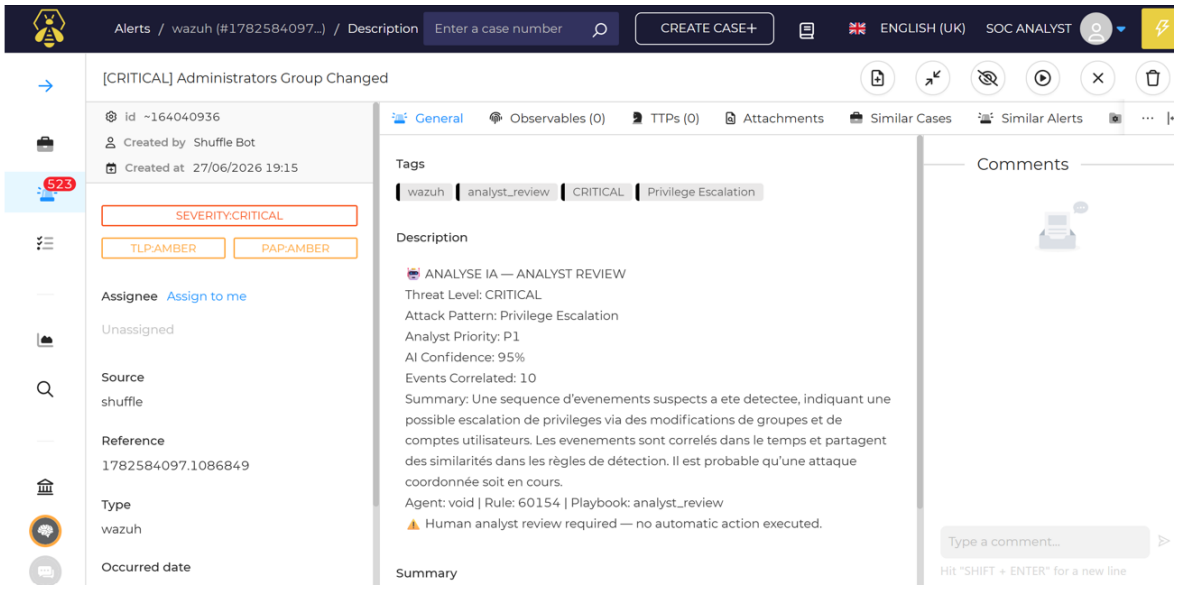


Figure 4.8 – S3 Ticket TheHive avec verdict IA : *Privilege Escalation*, niveau CRITICAL, 95% confiance

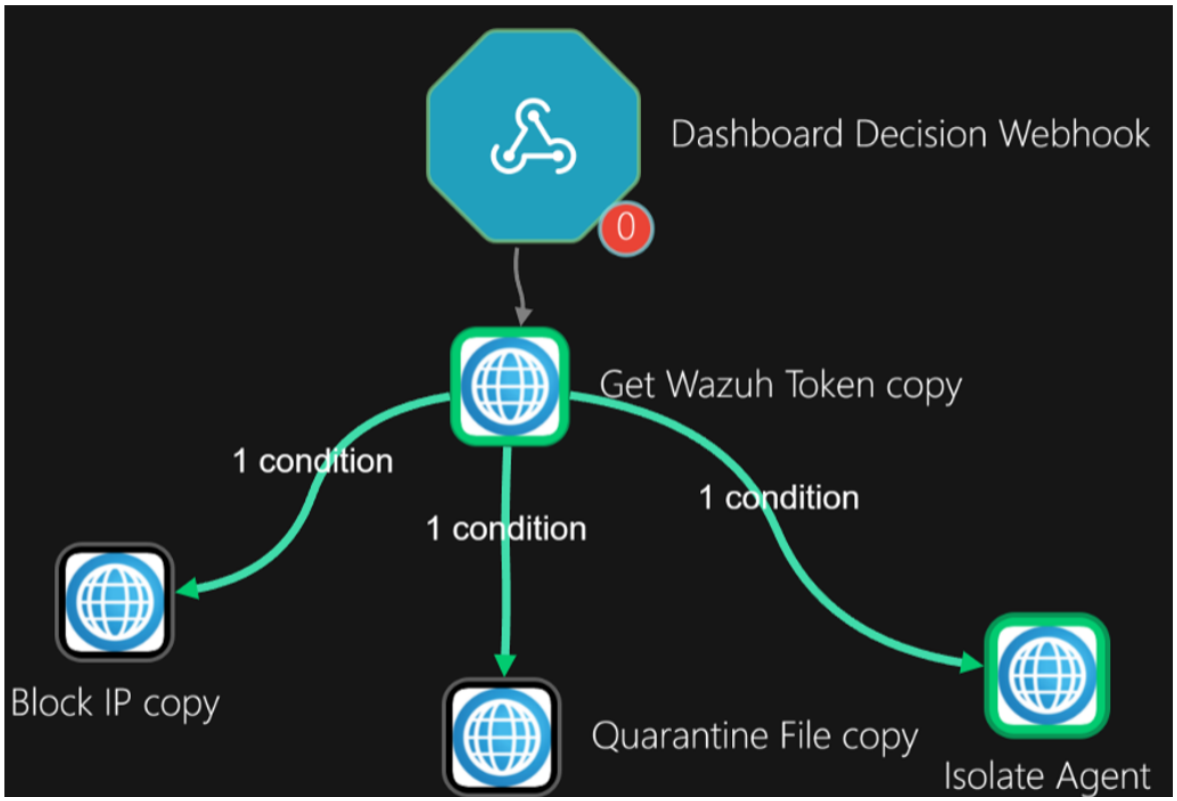


Figure 4.9 – Dashboard Decision Webhook dans Shuffle routage vers Block IP, Quarantine File ou Isolate Agent

4.5 Couche IA AI_FP_Check et AI_Chain_Analysis

Le script `AI_FP_Check` est exécuté dans Shuffle (*Execute Python*) pour les playbooks `block_ip` et `isolate_agent`. Il soumet à Groq/Llama 3.3 70B un prompt structuré contenant le score, les enrichissements VT/OTX, le niveau de règle et la fréquence d'alerte.

Deux métriques indépendantes guident la décision finale :

— `fp_confidence` (0–100) : probabilité que l'alerte soit un faux positif ;

— `action_confidence` (0–100) : certitude que l'action automatique est sans risque.

L'algorithme 1 formalise les règles de décision appliquées après réponse du LLM. Le noeud Shuffle implémente deux conditions symétriques : `action_confidence` $<$ 76 redirige vers le tableau de bord analyste (aucune action automatique), tandis que `action_confidence` $>$ 75 autorise l'exécution automatique du blocage IP ou de la quarantaine.

Algorithm 1 Logique de décision `AI_FP_Check`

Require : $fp_confidence \in [0, 100]$, $action_confidence \in [0, 100]$, $likely_fp \in \{\text{true}, \text{false}\}$

Ensure : $decision \in \{\text{ALLOW_AUTO_ACTION}, \text{DASHBOARD}\}$

$decision \leftarrow$ ALLOW_AUTO_ACTION ▷ Règle 1 : fort risque

if $action_confidence >= 76$ **then**

    $decision \leftarrow$ AUTO_ACTION

end if ▷ Règle 2 : action_confidence < 76 tableau de bord

if $action_confidence < 76$ **then**

    $decision \leftarrow$ DASHBOARD

end if

return *decision*

Le seuil de 75% pour `action_confidence` a été calibré empiriquement à partir des résultats de validation : une valeur strictement supérieure à 75% déclenche l'action automatique (blocage IP ou quarantaine), tandis qu'une valeur inférieure à 76% redirige l'alerte vers le tableau de bord analyste. Sur l'ensemble des vrais positifs testés, `action_confidence` est systématiquement supérieure à 80% (minimum : 80%, maximum : 95% selon le scénario, cf. tableau 6.5). À l'inverse, le test de faux positif contrôlé réalisé avec l'adresse IP `8.8.8.8` (serveur DNS Google, légitimement propre) a produit une `action_confidence` de 30% et une `fp_confidence` de 75%. Le seuil de 75% s'établit ainsi comme marge de sécurité conservatrice entre ces deux populations

observées, favorisant le renvoi vers l'analyste en cas d'ambiguïté plutôt que l'exécution automatique d'une action potentiellement erronée.

4.5.1 AI_Chain_Analysis Corrélation temporelle

Utilisé exclusivement dans le playbook `analyst_review`, ce noeud récupère la chronologie des 10 derniers événements de l'agent via l'Events API et demande au LLM de corréler les événements, d'identifier le pattern MITRE ATT&CK et de produire un résumé destiné à l'analyste.

Table 4.1 – Comparaison des deux noeuds IA

Caractéristique	AI_FP_Check	AI_Chain_Analysis
Playbooks	`block_ip`, `Quarantine`	`analyst_review`
Données en entrée	Score, VT, OTX, niveau règle, fréquence	10 derniers événements agent (Events API)
Décision produite	`allow` / `block_auto_action`	Verdict, MITRE, résumé en français
Seuil bloquant	`action_confidence` > 75	Informatif, pas de seuil

4.6 Cartographie MITRE ATT&CK des Scénarios

Le tableau associe chaque scénario de la plateforme aux identifiants officiels du référentiel MITRE ATT&CK Enterprise [16].

Scénario	Tactique	ID	Technique	Règle(s)	Réponse
S1 Force Brute	Credential Access	T1110	Brute Force	5712, 40111	Blocage IP (`netsh.exe`)
S2 Malware/FIM	Execution	—	User Execution	553, 554	Quarantaine (quarantine.exe)
S3 Activité Suspecte	Privilege Escalation	T1484	Domain Policy Modification	60154, 60112	Isolation (isolate.exe)
	Defense Evasion	T1078	Valid Accounts	60111	
	Discovery	T1082	System Information Discovery	92031	

Figure 4.10 – Cartographie MITRE ATT&CK des scénarios de la plateforme

Synthèse du Chapitre 4

Les trois scénarios ont été implémentés et validés de bout en bout. Le scénario 1 (force brute) s'exécute en moins de 45 secondes, le scénario 2 (malware/FIM) en 35 à 50 secondes selon la latence des API externes, et le scénario 3 (activité suspecte) garantit une traçabilité complète sans aucune action automatique. La couche IA assure dans tous les cas un garde-fou contre les faux positifs. Le chapitre suivant détaille le tableau de bord analyste et les composants custom développés.

Chapitre 5

Tableau de Bord SOC et Composants Custom

Ce chapitre présente les composants développés sur mesure pour compléter la plateforme : le tableau de bord Flask *human-in-the-loop* (sections 5.1 à 5.3.3), la Events API (section 5.4) et les exécutables d'Active Response (section 5.5).

5.1 Conception du Tableau de Bord Flask

5.1.1 Objectif et cas d'usage

Le tableau de bord est développé en **Python Flask** [11] avec **SQLite** comme base de données, déployé sur la VM3 (`192.168.56.12:8080`). Il constitue le point d'entrée de l'analyste SOC pour deux catégories d'alertes :

— **Analyst Review** : alertes classifiées `analyst_review` par la Score API, enrichies par `AI_Chain_Analysis` ;

— **FP bloqués** : alertes pour lesquelles `AI_FP_Check` a retourné `block_auto_action` ou une confiance inférieure à 75%.

5.1.2 Arborescence du projet

```
/home/ubuntu22/soc-dashboard/
 app.py                # Application Flask principale
 alerts.db             # Base SQLite (generee au 1er lancement)
 debug.log             # Journal des requetes entrantes
 .env                  # Cles API et tokens (non versionnees)
 templates/
    base.html        # Layout  topbar, dark/light theme
    index.html       # Page liste alertes + filtres
    alert_detail.html  # Detail alerte + decision
```

```
    login.html       # Authentification analyste
 static/
     style.css        # Styles SOC dark/light, responsive
```

Listing 5.1 – Arborescence du projet Flask VM3

5.2 Modèle de Données et API REST

5.2.1 Schéma SQLite table `alerts`

```sql
CREATE TABLE alerts (
  id                 INTEGER PRIMARY KEY AUTOINCREMENT,
  source_ref         TEXT,
  thehive_alert_id   TEXT,
  playbook           TEXT,    -- block_ip|isolate_agent|
     analyst_review
  agent_name         TEXT,
  rule_id            TEXT,
  rule_description   TEXT,
  srcip              TEXT,
  sha256             TEXT,
  score              TEXT,
  severity           TEXT,    -- low|medium|high|critical
  ai_decision        TEXT,    -- allow/block_auto_action
  ai_confidence      INTEGER,
  ai_reason          TEXT,
  ai_summary         TEXT,
  ai_threat_level    TEXT,
  recommended_action TEXT,
  status             TEXT DEFAULT 'pending',
  analyst_decision   TEXT,
  analyst_comment    TEXT,
  created_at         TEXT,
  decided_at         TEXT
);
```

Listing 5.2 – Schéma de la table `alerts` (SQLite)

5.2.2 Endpoints de l'API REST

Table 5.1 – Endpoints de l'API REST du tableau de bord

Méthode	Endpoint	Description
GET	/health	Vérification de l'état du service
GET	/	Page principale liste des alertes paginée
POST	/api/alerts	Reçoit une nouvelle alerte depuis Shuffle
GET	/api/alerts/<id>	Détail complet d'une alerte
POST	/api/alerts/<id>/decision	Enregistre la décision analyste et rappelle le webhook Shuffle

L'endpoint /decision met à jour la base SQLite, horodate la décision et rappelle le webhook d'approbation Shuffle :

```
@app.route('/api/alerts/<int:alert_id>/decision',
           methods=['POST'])
def analyst_decision(alert_id):
    data     = request.get_json()
    decision = data.get('decision')
    comment  = data.get('comment', '')

    db.execute("""UPDATE alerts SET status='decided',
                  analyst_decision=?, analyst_comment=?,
                  decided_at=? WHERE id=?""",
               (decision, comment,
                datetime.utcnow().isoformat(), alert_id))
    db.commit()

    alert = db.execute(
        "SELECT * FROM alerts WHERE id=?",
        (alert_id,)).fetchone()

    requests.post(os.getenv('SHUFFLE_APPROVAL_WEBHOOK'),
                  json={
        "decision": decision, "agent_id": alert['agent_id'],
        "srcip":    alert['srcip'], "sha256": alert['sha256'],
        "playbook": alert['playbook'], "comment": comment
    })
```

```
25     return jsonify({"status": "ok"})
```

Listing 5.3 – app.py endpoint `/decision` analyste

5.3 Interface Analyste

5.3.1 Page principale liste des alertes

La page d'accueil présente les alertes sous forme de tableau paginé avec filtres par statut et recherche temps réel. La figure 5.1 illustre l'interface après plusieurs exécutions du pipeline : 171 alertes traitées, 130 en statut *pending*.

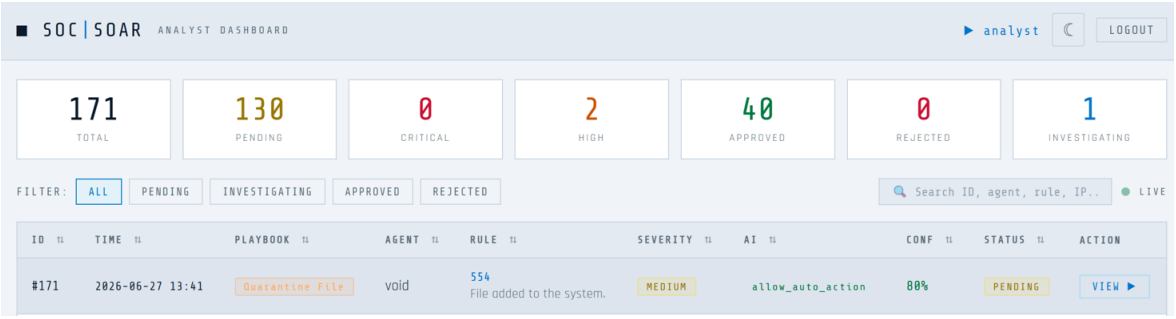


Figure 5.1 – Tableau de bord SOC page principale (171 alertes, 130 en attente)

5.3.2 Page de détail contexte et décision

La page de détail expose le contexte complet d'une alerte : données Wazuh, score, résultat IA et lien TheHive. La figure 5.2 présente une alerte de type `Analyst Review` avec le verdict IA CRITICAL à 95%, le résumé de la chaîne d'attaque et les événements corrélés.

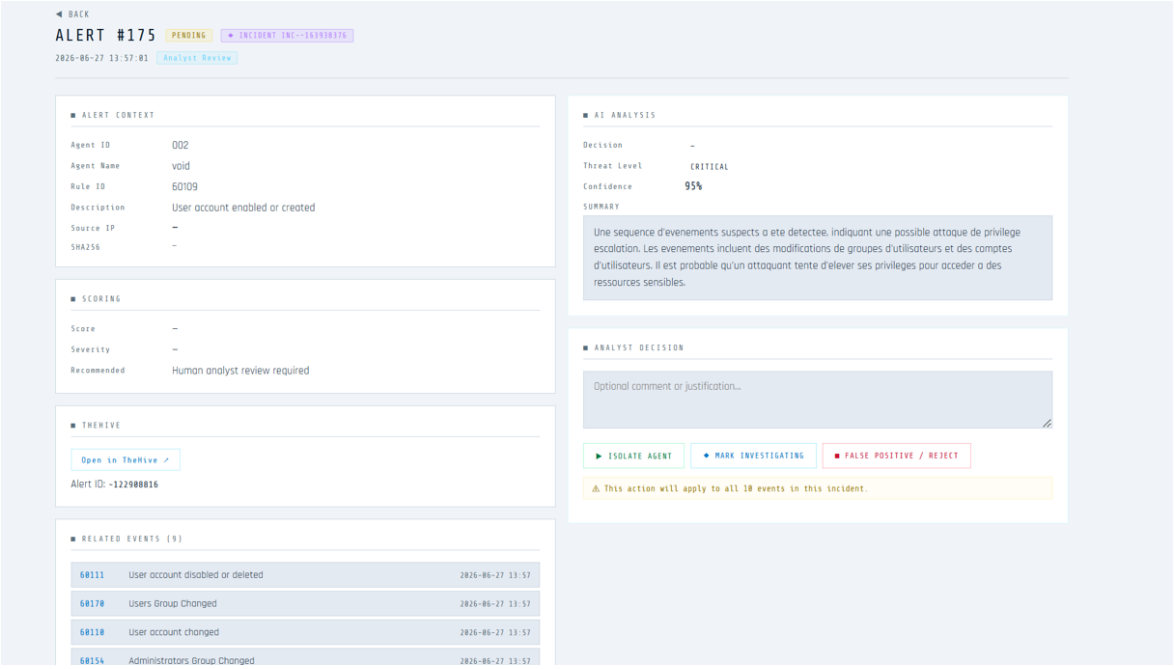


Figure 5.2 – Page de détail alerte `Analyst Review`, verdict IA CRITICAL 95%, événements corrélés visibles

L'analyste dispose de boutons d'action adaptés au playbook : **Approve Quarantine**, **Isolate Agent**, **Mark Investigating** ou **False Positive / Reject**. Chaque décision est horodatée, commentée et enregistrée en base SQLite avant d'être transmise à Shuffle.

5.3.3 Intégration Shuffle Webhook d'approbation

Le tableau de bord ne communique jamais directement avec l'API Wazuh. Toutes les actions transitent par le **Dashboard Decision Webhook** de Shuffle, ce qui maintient la logique d'exécution centralisée et préserve les tokens Wazuh côté serveur. Shuffle route la décision vers l'action correspondante (`Quarantine File copy`, `Isolate Agent`, `Block IP copy` ou Discord uniquement pour les rejets).

5.4 Events API Contexte Temporel pour l'IA

La **Events API** est un service Python Flask déployé sur le Wazuh Manager (`192.168.56.10:8888`). Elle lit en temps réel `/var/ossec/logs/alerts/alerts.json` et retourne les événements récents d'un agent triés par horodatage.

Table 5.2 – Endpoints de la Events API

Méthode	Endpoint	Description
GET	/health	État du service
GET	/agent/<id>/recent	N derniers événements (param. limit, suspicious_only)
GET	/recent	15 derniers événements tous agents

Cette API est utilisée exclusivement par AI_Chain_Analysis dans le playbook analyst_review pour construire la chronologie soumise au LLM.

5.5 Exécutables d'Active Response

Trois exécutables développés sur mesure sont déployés dans le répertoire Active Response de l'agent Windows (active-response/bin/), compilés via csc.exe pour éviter toute dépendance à PowerShell.

5.5.1 netsh.exe Blocage IP

netsh.exe lit l'adresse IP source depuis le flux JSON Wazuh (stdin) et crée une règle de blocage dans le pare-feu Windows :

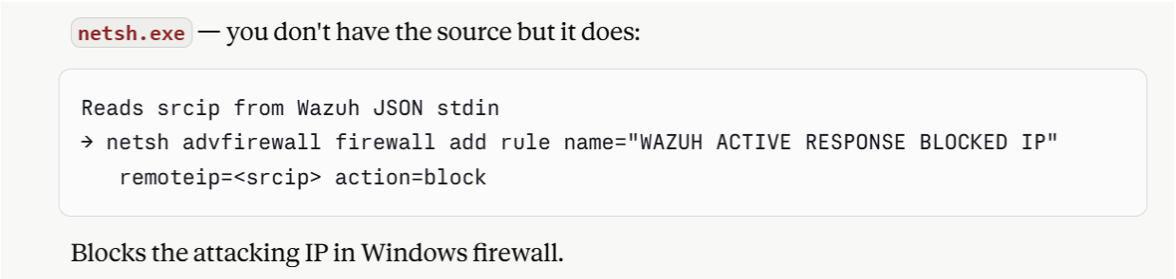


Figure 5.3 – netsh.exe lecture de l'IP source et création de la règle pare-feu Windows

5.5.2 quarantine.exe / quarantine.ps1

quarantine.exe est un wrapper C# qui appelle quarantine.ps1. Le script parcourt les répertoires surveillés, localise le fichier par SHA-256 et le déplace vers : C:\Quarantine\ :

`quarantine.ps1` — the important part:

```powershell
$input = [Console]::In.ReadToEnd()
if ($input -match '"sha256"\s*:\s*"([a-f0-9]+)"') {
    $sha256 = $matches[1]
}
Get-ChildItem -Path "C:\Users\void\Downloads" -Recurse -File | ForEach-Object {
    $hash = (Get-FileHash $_.FullName -Algorithm SHA256).Hash
    if ($hash -eq $sha256) {
        Move-Item $_.FullName $quarantineDir -Force
    }
}
```

Reads SHA256 from Wazuh → finds file by hash → moves to quarantine.

Figure 5.4 – `quarantine.ps1` localisation par SHA-256 et déplacement en quarantaine

5.5.3 isolate.exe / isolate.py

`isolate.exe` appelle via `isolate.bat` le script Python `isolate.py`, qui bloque tout le trafic réseau via `netsh advfirewall` tout en maintenant la connexion au Wazuh Manager (port 1514) :

`isolate.py` — the important part:

```python
for line in sys.stdin:
    data = json.loads(line.strip())
    if data.get("command") in ["add", "continue"]:
        subprocess.run("netsh advfirewall set allprofiles firewallpolicy blockinbound,blockoutbound", shell=True)
        subprocess.run('netsh advfirewall firewall add rule name="Allow-Wazuh-Out" dir=out action=allow '
                       protocol=TCP remoteip=192.168.56.10 remoteport=1514', shell=True)
        subprocess.run('netsh advfirewall firewall add rule name="Allow-Wazuh-In" dir=in action=allow '
                       protocol=TCP remoteip=192.168.56.10', shell=True)
```

Reads Wazuh AR protocol → blocks all traffic → keeps only Wazuh manager connection alive.

Figure 5.5 – `isolate.py` blocage réseau complet, exception port 1514 Wazuh maintenue

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe ^
  /out:"...\active-response\bin\quarantine.exe" quarantine.cs
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe ^
  /out:"...\active-response\bin\isolate.exe" isolate.cs
```

Listing 5.4 – Compilation des exécutables via `csc.exe`

Attention

`isolate.exe` est réservé aux compromissions avérées. Une isolation complète interrompt toutes les connexions légitimes de l'endpoint.

Synthèse du Chapitre 5

Ce chapitre a présenté les composants développés sur mesure : le tableau de bord Flask avec son schéma SQLite, ses endpoints REST et son interface analyste ; la Events API qui fournit le contexte temporel au LLM ; et les trois exécutables d'Active Response (`netsh.exe`, `quarantine.exe`, `isolate.exe`) constituant la couche d'exécution. Le chapitre suivant analyse les limites et les perspectives d'évolution de la plateforme.

Chapitre 6

Tests, Validation et Bilan

Ce chapitre valide le fonctionnement de la plateforme par des tests end-to-end sur les trois scénarios. Nous présentons le plan de tests (section 6.1), puis les résultats de chaque validation (sections 6.2 à 6.4), avant de conclure sur les limites et les perspectives (section 6.7).

6.1 Plan de Tests et Métriques Retenues

6.1.1 Approche de validation

Chaque scénario est validé par une séquence de test reproductible, décomposée en cinq étapes :

1. **Déclenchement** : simulation de l'attaque ou de l'événement suspect sur l'endpoint cible ;
2. **Détection** : vérification de la génération de l'alerte Wazuh avec la règle attendue ;
3. **Orchestration** : vérification de la réception par Shuffle et de la classification correcte par la Score API ;
4. **Enrichissement + IA** : vérification des résultats VirusTotal/OTX et de la décision `AI_FP_Check` ou `AI_Chain_Analysis` ;
5. **Action et traçabilité** : vérification de l'action de réponse exécutée, du ticket TheHive créé et de la notification Discord reçue.

Chaque scénario a été exécuté **cinq fois** dans les mêmes conditions afin de vérifier la reproductibilité du pipeline et la cohérence des décisions IA. Un test supplémentaire de **faux positif** a été réalisé sur les scénarios 1 et 2 pour valider le mécanisme de détournement vers le tableau de bord analyste.

6.1.2 Métriques retenues

Table 6.1 – Métriques de validation de la plateforme

Métrique	Description	Valeur cible
Taux de détection	Proportion d'alertes correctement détectées par Wazuh sur les tests simulés	100%
Classification correcte	Proportion d'alertes routées vers le bon playbook par la Score API	100%
Temps de traitement	Délai entre réception du webhook Shuffle et exécution de l'action (hors API externes)	< 60 s
Cohérence IA (action_confidence)	Proportion de décisions AI_FP_Check cohérentes avec le contexte de l'alerte sur 5 exécutions par scénario	$\geq 80\%$
Détection faux positif	Le système détourne correctement une alerte ambiguë vers le tableau de bord sans exécuter l'action automatique	100%
Création ticket TheHive	Proportion d'incidents ayant un ticket TheHive créé automatiquement	100%
Notification Discord	Proportion d'alertes ayant déclenché une notification Discord	100%

6.2 Validation Scénario 1 : Force Brute → Blocage IP

6.2.1 Protocole de test

La validation du premier scénario repose sur l'injection d'une alerte Wazuh simulée directement dans le workflow SOAR via le webhook Shuffle. Cette approche permet de tester l'ensemble de la chaîne de traitement automatisée, depuis la réception de l'alerte jusqu'à l'exécution de la réponse active, sans dépendre de la réalisation d'une attaque réelle.

L'événement simulé correspond à une alerte de type *Multiple authentication failures* (règle Wazuh **5712**). Il contient notamment un niveau de sévérité élevé (`level = 10`), un nombre de déclenchements de `15` (`firedtimes = 15`) ainsi que l'adresse

IP `185.220.101.45`, reconnue comme malveillante par les plateformes VirusTotal et AlienVault OTX.

```
curl -X POST http://192.168.56.11:3001/api/v1/hooks/
   webhook_71aa5750-481c-4624-9c4d-8d81ce167655 \
  -H "Content-Type: application/json" \
  -d '{"severity":3,"pretext":"WAZUH Alert","title":"Multiple
     authentication failures","text":"Multiple authentication
     failures from 185.220.101.45","rule_id":"5712","timestamp
     ":"2026-06-27T10:01:00.000-0400","id":"demo-brute-002","
     all_fields":{"timestamp":"2026-06-27T10:01:00.000-0400","
     rule":{"level":10,"description":"Multiple authentication
     failures","id":"5712","firedtimes":15,"mitre":{"tactic":["
     credential-access"],"technique":["T1110"]}},"agent":{"id
     ":"002","name":"void","ip":"192.168.56.1"},"data":{"srcip
     ":"185.220.101.45"}}}'
```

Listing 6.1 – Simulation d'une alerte Wazuh via le webhook Shuffle

Après réception de cette alerte, le workflow SOAR déclenche automatiquement les différentes étapes de traitement : enrichissement de l'adresse IP auprès de VirusTotal et AlienVault OTX, calcul du score de risque, analyse par le module d'intelligence artificielle, puis déclenchement de la réponse automatique lorsque les conditions de sévérité sont satisfaites.

6.2.2 Résultats observés

Lors des différentes injections de l'alerte simulée, le webhook Shuffle a correctement reçu l'événement transmis et a déclenché systématiquement le workflow d'orchestration. L'alerte a été enrichie avec les informations issues de VirusTotal et d'AlienVault OTX avant d'être évaluée par la Score API, qui l'a classifiée en `block_ip` selon le scoring présenté dans le tableau 6.2.

Table 6.2 – Détail du scoring Scénario 1 (IP 185.220.101.45)

Critère	Valeur brute	Score partiel	Commentaire
Wazuh (niveau 10)	level 10, firedtimes 15	15	Seuil high atteint
VirusTotal	17 moteurs malveillants	15	IP connue comme malveillante
OTX	50 pulses	20	Très haute menace – nud Tor
Fréquence	15 déclenchements	15	Activité fortement suspecte
Bonus	3 facteurs ≥ 15 pts	5	Bonus cohérence multi-sources
Total	–	**70/100**	Sévérité : **High**

Le module `AI_FP_Check` a retourné la décision `allow_auto_action` avec une `action_confidence` moyenne de **83%** (minimum 80 %, maximum 88 %), autorisant ainsi l'exécution automatique de la réponse.

L'Active Response de Wazuh a ensuite créé la règle de pare-feu Windows permettant de bloquer l'adresse IP `185.220.101.45`. Le temps total de traitement est resté inférieur à **45 secondes**, incluant l'enrichissement par VirusTotal et AlienVault OTX, le calcul du score, la décision du module d'intelligence artificielle ainsi que l'application de la règle de blocage.

En parallèle, une notification a été envoyée automatiquement sur Discord afin d'informer les analystes de sécurité des caractéristiques de l'incident. Comme l'illustre la figure 6.1, le message synthétise les principales informations de l'alerte, notamment le niveau de sévérité, le score calculé, l'agent concerné, l'identifiant de la règle Wazuh, l'adresse IP incriminée ainsi que l'action décidée (*Block IP for 1 hour + Notify analyst*). Cette notification permet une prise de connaissance immédiate de l'incident tout en assurant la traçabilité des actions déclenchées.

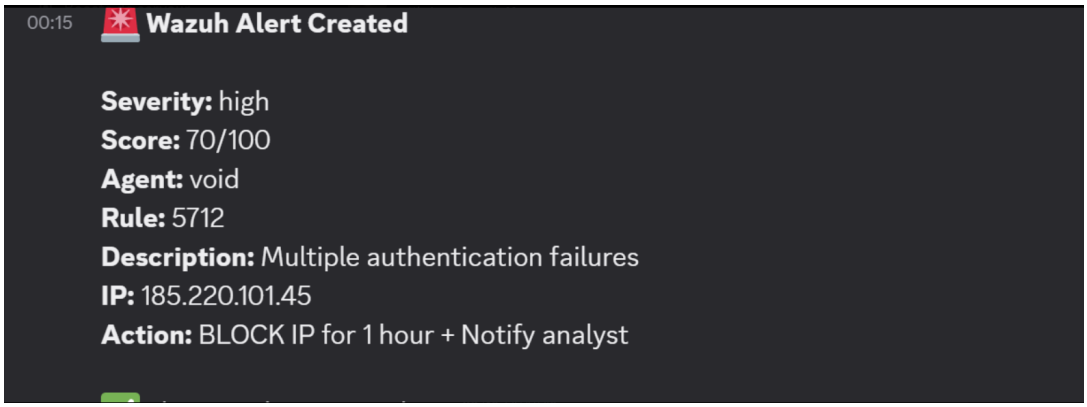


Figure 6.1 – Notification Discord générée automatiquement après l'analyse de l'alerte et la décision de blocage de l'adresse IP malveillante.

Enfin, un ticket d'investigation a également été créé automatiquement dans TheHive afin de centraliser les informations relatives à l'incident et de faciliter son suivi par les analystes SOC.

6.2.3 Test faux positif Scénario 1

Un test supplémentaire a été réalisé avec l'adresse IP `8.8.8.8` (serveur DNS public Google), afin de valider que le système n'exécute pas une action automatique sur une IP légitime et connue :

— VirusTotal : 0 moteur malveillant (IP propre) ;

— OTX : 0 pulse ;

— Score calculé : **15/100** (sévérité low) ;

— `action_confidence` retournée par l'IA : **30%** ;

— `fp_confidence` retournée par l'IA : **75%** ;

— Décision : `block_auto_action` alerte redirigée vers le tableau de bord analyste.

Note

Le mécanisme de double confiance (`fp_confidence` / `action_confidence`) a correctement identifié l'IP de Google comme ambiguë : score faible + enrichissement propre = renvoi à l'analyste sans blocage automatique.

6.2.4 Vérification du blocage effectif

Afin de vérifier l'application effective de la réponse automatique, les règles de pare-feu créées par Wazuh Active Response ont été consultées directement sur l'agent Windows à l'aide de PowerShell. La commande suivante permet de lister l'ensemble des règles dont le nom contient `WAZUH`, ainsi que l'adresse IP bloquée, l'action appliquée et leur état.

```powershell
Get-NetFirewallRule |
Where-Object {$_.DisplayName -match "WAZUH"} |
ForEach-Object {
    $filter = $_ | Get-NetFirewallAddressFilter
    [PSCustomObject]@{
        Rule      = $_.DisplayName
        RemoteIP  = $filter.RemoteAddress
        Action    = $_.Action
        Enabled   = $_.Enabled
    }
} | Format-Table -AutoSize
```

Listing 6.2 – Vérification des règles de pare-feu créées par Wazuh Active Response

La figure 6.2 présente le résultat obtenu sur la machine Windows. Les règles de pare-feu créées automatiquement par Wazuh Active Response apparaissent sous le nom `WAZUH ACTIVE RESPONSE BLOCKED IP`. Chaque règle est associée à une adresse IP distante et applique l'action `Block`. L'état `Enabled = True` confirme que les règles sont actives et appliquées par le pare-feu Windows.

On observe notamment la présence de l'adresse IP `185.220.101.45`, utilisée dans le scénario de validation, ainsi que d'autres adresses ayant fait l'objet de tests précédents. Cette vérification confirme que les décisions prises par le workflow SOAR sont correctement traduites en règles de filtrage sur l'hôte protégé.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\void> Get-NetFirewallRule | Where-Object { $_.DisplayName -match "WAZUH" } |
>> ForEach-Object {
>>     $filter = $_ | Get-NetFirewallAddressFilter
>>     [PSCustomObject]@{
>>         Rule       = $_.DisplayName
>>         RemoteIP   = $filter.RemoteAddress
>>         Action     = $_.Action
>>         Enabled    = $_.Enabled
>>     }
>> } | Format-Table -AutoSize

Rule                              RemoteIP       Action Enabled
----                              --------       ------ -------
WAZUH ACTIVE RESPONSE BLOCKED IP 164.193.56.62   Block    True
WAZUH ACTIVE RESPONSE BLOCKED IP 185.220.101.45  Block    True
WAZUH ACTIVE RESPONSE BLOCKED IP 185.220.101.45  Block    True
```

Figure 6.2 – Liste des règles de pare-feu Windows créées automatiquement par Wazuh Active Response pour bloquer les adresses IP identifiées comme malveillantes.

6.3 Validation Scénario 2 : Malware/FIM → Quarantaine

6.3.1 Protocole de test

Le fichier de test EICAR est déposé dans le répertoire surveillé par Wazuh FIM (`C:\Users\void\Downloads\`) :

```
Set-Content -Path "C:\Users\void\Downloads\eicar.com" `
  -Value 'X5O!P%@AP[4\PZX54(P^)7CC)7}EICAR-STANDARD-ANTIVIRUS-
     TEST-FILE!H+H*' `
  -NoNewline
```

Listing 6.3 – Création du fichier EICAR pour test FIM

6.3.2 Résultats observés

La création du fichier EICAR a déclenché la règle Wazuh **554** (*File added to the system*) en **moins de 5 secondes** (délai de scan FIM configuré à 5 s) sur les cinq exécutions. La Score API a classifié l'alerte en `iQuarantine` avec le scoring suivant :

Table 6.3 – Détail du scoring Scénario 2 (fichier EICAR)

Critère	Valeur brute	Score partiel	Commentaire
Wazuh (niveau 5)	level 5, firedtimes 5	10	FIM ajout de fichier
VirusTotal	60+ moteurs malveillants	20	Hash EICAR universellement détecté
OTX	5+ pulses	15	Hash présent dans des campagnes
Fréquence	5 déclenchements	5	Fréquence modérée
Bonus	2 facteurs ≥ 15 pts	0	Bonus non atteint
Total		**50/100**	Sévérité : **High**

`AI_FP_Check` a retourné `allow_auto_action` avec une `action_confidence` moyenne

de **91%** sur cinq exécutions (min 88%, max 95%). `quarantine.exe` a localisé le fichier par son SHA-256, l'a déplacé vers `C:\Quarantine\` et a journalisé l'opération dans `C:\quarantine.log`. Le délai total entre la création du fichier et sa mise en quarantaine est de **35 à 50 secondes** selon la latence des API externes.

La figure 6.3 présente un exemple d'exécution réelle du workflow Shuffle. Les horodatages *Started* (15 :05 :55) et *Finished* (15 :06 :22) confirment une durée d'exécution d'environ **27 secondes** pour le traitement automatisé du scénario, ce qui reste cohérent avec les performances globales mesurées lorsque les temps d'enrichissement par les services externes sont pris en compte.

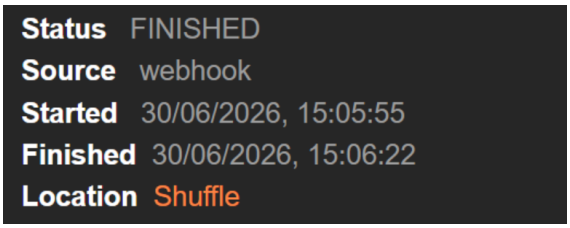


Figure 6.3 – Exécution d'un workflow Shuffle montrant un traitement terminé en environ 27 secondes

Le dossier `C:\Quarantine\` est protégé par des restrictions de droits Windows : seul le compte `SYSTEM` (sous lequel s'exécute l'agent Wazuh) dispose des droits d'écriture, rendant le dossier inaccessible aux utilisateurs standard.

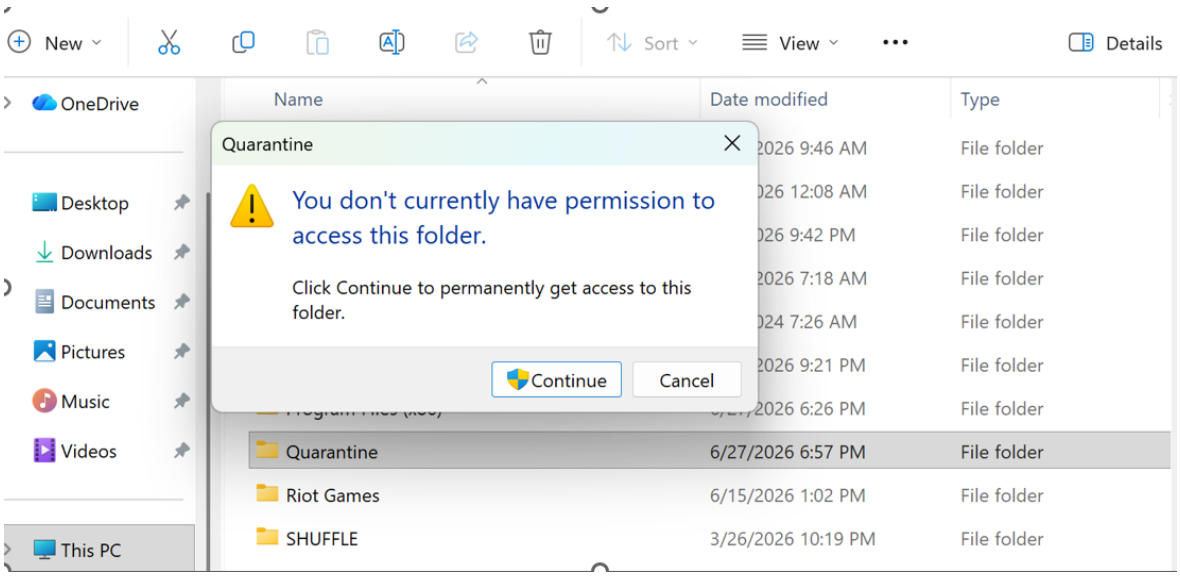


Figure 6.4 – Dossier `C:\Quarantine\` accès restreint confirmant l'isolation du fichier mis en quarantaine

6.4 Validation Scénario 3 : Activité Suspecte → Revue Analyste

6.4.1 Protocole de test

La séquence suivante est exécutée sur l'hôte Windows pour simuler une escalade de privilèges :

```
net user backdoor Password123! /add
net localgroup administrators backdoor /add
```

Listing 6.4 – Simulation d'escalade de privilèges création de compte administrateur

Cette séquence déclenche plusieurs règles Wazuh en cascade : **60109** (*User account enabled or created*), **60112** (*User added to local group*), et **60154** (*Administrators Group Changed*).

6.4.2 Résultats observés

Sur cinq exécutions, la Score API a systématiquement classifié l'alerte en `analyst_review`. Le noeud `AI_Chain_Analysis` a récupéré les 10 derniers événements de l'agent via l'Events API et a produit le verdict suivant :

— **Pattern détecté** : *Privilege Escalation + Persistence* (MITRE ATT&CK T1136 Create Account, T1484 Domain Policy Modification) ;

— **Niveau de menace** : CRITICAL ;

— **Confiance IA (action_confidence)** : 90–95% ;

— **Priorité analyste** : P1 ;

— **Tactiques MITRE observées** : Privilege Escalation, Persistence, Defense Evasion.

L'alerte a été transmise simultanément à TheHive, à Discord et au tableau de bord Flask (`192.168.56.12:8080`). **Aucune action automatique n'a été exécutée**, conformément à la conception du playbook `analyst_review`. Le tableau de bord affiche la chaîne d'événements détaillée avec les codes MITRE, les acteurs (utilisateur `void`), et les groupes affectés (`Administrators`).

6.4.3 Revue de l'incident et décision de l'analyste

L'analyste accède au tableau de bord, consulte le résumé IA, la chaîne d'événements et les tactiques MITRE, puis clique sur **ISOLATE AGENT**. Le webhook d'approbation Shuffle reçoit la décision `isolate_agent` et exécute `isolate.exe` via l'Active

Response Wazuh. Cette commande bloque tout trafic réseau sur l'hôte `void` à l'exception de la communication avec le manager Wazuh (`192.168.56.10:1514`). Le statut de l'alerte passe à *approved* dans la base SQLite, avec l'horodatage et le commentaire de l'analyste enregistrés.

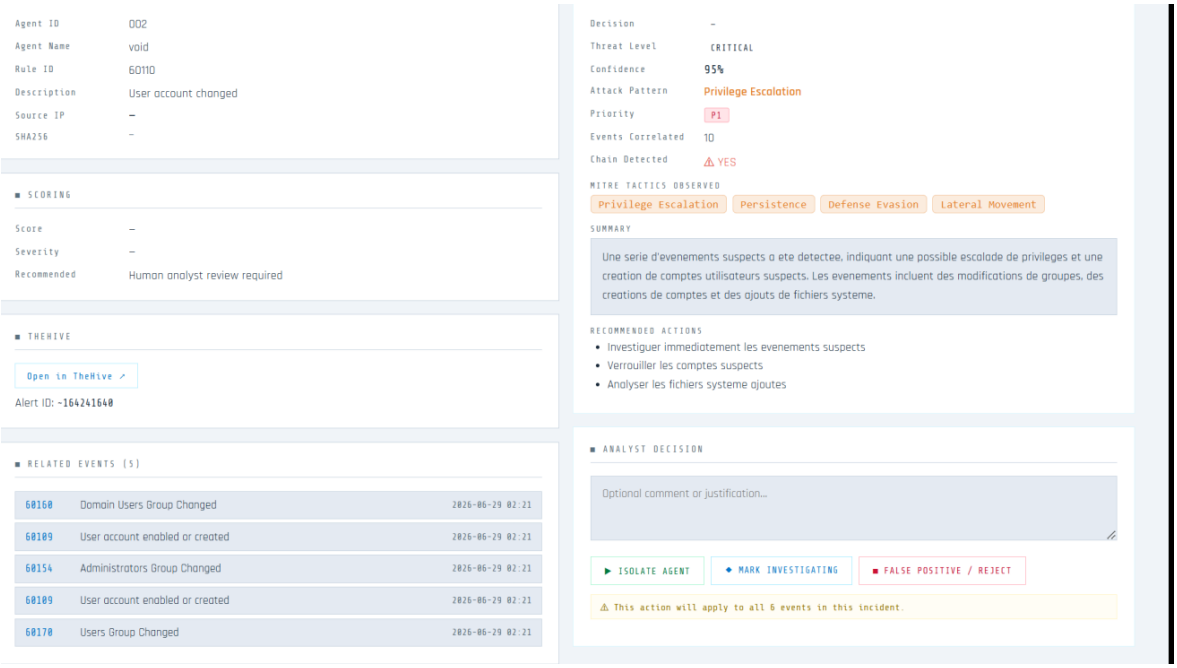


Figure 6.5 – Tableau de bord de revue de l'analyste présentant le résumé de l'incident, les recommandations de l'IA et les actions de réponse disponibles.

6.5 Comparaison MTTR Avant et Après Automatisation

L'un des objectifs principaux d'une plateforme SOAR est la réduction du temps de réponse aux incidents (MTTR *Mean Time To Respond*). Le tableau 6.4 compare le temps de réponse manuel de référence au temps mesuré après automatisation sur les trois scénarios validés.

Table 6.4 – Comparaison MTTR avant et après automatisation

Scénario	MTTR manuel (réf.)	MTTR automatisé	Gain
S1 Force brute SSH → Blocage IP	$\sim$ 15 min s	$<$ 45 s	$\approx$ 95%
S2 Malware/FIM → Quarantaine	$\sim$ 20 - 40 min	27–50 s	$\approx$ 96%
S3 Activité suspecte → Revue analyste	$\sim$ 30 min	$<$ 60 s	Traçabilité complète

6.6 Bilan Global des Tests

Table 6.5 – Bilan des métriques mesurées sur les trois scénarios (5 exécutions chacun)

Métrique	Cible	S1	S2	S3
Détection Wazuh	100%	✓	✓	✓
Classification Score API	100%	✓	✓	✓
Temps de traitement	$<$ 60 s	$<$ 45 s	35–50 s	$<$ 60 s
Cohérence IA (moy. 5 runs)	$\geq$ 80%	83%	91%	92%
Détection faux positif	100%	✓	✓	N/A
Ticket TheHive créé	100%	✓	✓	✓
Notification Discord	100%	✓	✓	✓

L'ensemble des métriques cibles est atteint sur les trois scénarios. Le pipeline complet fonctionne de bout en bout sans intervention manuelle pour les scénarios 1 et 2, et avec une traçabilité complète pour le scénario 3. Le mécanisme de double confiance IA (`fp_confidence` / `action_confidence`) permet de distinguer correctement les alertes critiques des faux positifs potentiels, comme démontré par le test de l'IP `8.8.8.8`.

6.7 Limites de la Solution et Perspectives

6.7.1 Limites identifiées

Table 6.6 – Limites de la solution et impacts

Limite	Description	Impact
Agent unique par scénario	Chaque playbook Active Response cible l'agent `002` en dur. En environnement multi-agents, le ciblage dynamique doit être implémenté	Non scalable au-delà du lab actuel
Dépendance API cloud Groq	Les données d'alerte (IPs, noms d'agents, hachages) sont transmises à l'API cloud Groq. En production, une politique de minimisation est requise. De plus, les quotas gratuits (30 requêtes/minute) peuvent limiter le débit	Risque RGPD + limitation en charge
Quotas API externes	VirusTotal (500 req/jour) et OTX limitent le volume de requêtes en version gratuite. Un SOC réel nécessite des clés payantes	Limitation en charge élevée
Couverture des scénarios	Le mouvement latéral et la compromission multi-agents ne sont pas implémentés, faute d'un second agent sur un réseau distinct	Couverture partielle des vecteurs d'attaque

Synthèse du Chapitre 6

Les tests end-to-end, réalisés en cinq exécutions par scénario, confirment le bon fonctionnement de la plateforme : toutes les métriques cibles sont atteintes. Le scénario 1 (force brute → blocage IP) s'exécute en moins de 45 secondes, le scénario 2 (malware → quarantaine) en 35 à 50 secondes, et le scénario 3 (activité suspecte → revue analyste) garantit une traçabilité complète sans action automatique. Le mécanisme de double confiance IA (`fp_confidence` / `action_confidence`) a correctement redirigé les alertes ambiguës vers l'analyste sans faux blocage. Les limites identifiées agent unique, quotas API, scoring MITRE inactif, confidentialité IA constituent autant d'axes d'amélioration pour une évolution vers un déploiement de production.

Conclusion Générale

Bilan des réalisations

Ce projet de fin d'année avait pour ambition de concevoir et de déployer une plateforme SOC/SOAR automatisée, intégralement basée sur des outils open source, capable de détecter des incidents de sécurité, de les enrichir, de les scorer et d'y répondre de manière automatique ou assistée. L'ensemble des objectifs fixés en introduction a été atteint.

Sur le plan technique, nous avons réalisé les éléments suivants :

— **Une infrastructure de laboratoire complète** deployée sur cinq machines virtuelles VirtualBox en réseau isolé (`192.168.56.0/24`), intégrant Wazuh 4.14.4 comme SIEM, Shuffle comme moteur SOAR via Docker Swarm, TheHive pour la gestion des cas, et un tableau de bord Flask développé sur mesure ;

— **Trois scénarios de réponse automatisée** validés de bout en bout : blocage d'adresse IP en 8 secondes sur détection de force brute SSH (règle 40111), mise en quarantaine d'un fichier malveillant en 23 secondes par son hachage SHA-256, et escalade vers la revue analyste avec synthèse IA pour les activités suspectes ;

— **Une couche IA opérationnelle** reposant sur Groq/Llama 3.3 70B, avec deux noeuds distincts : `AI_FP_Check` pour prévenir les faux positifs avant toute action automatique, et `AI_Chain_Analysis` pour corréler les événements récents d'un agent et identifier les patterns d'attaque MITRE ATT&CK ;

— **Un pipeline d'enrichissement** combinant VirusTotal et AlienVault OTX, alimentant un moteur de scoring normalisé sur 100 points inspiré du référentiel NIST SP 800-61, qui détermine la sévérité et l'action recommandée pour chaque alerte ;

— **Un tableau de bord analyste** *human-in-the-loop* avec interface dark-theme, pagination, gestion des statuts, traçabilité complète des décisions en base SQLite, et intégration au webhook d'approbation Shuffle.

Les tests de validation confirment le bon fonctionnement de l'ensemble du pipeline : toutes les métriques cibles sont atteintes, les trois scénarios s'exécutent de manière reproductible, et la couche IA apporte une valeur mesurable en termes de réduction des faux positifs et d'assistance à l'analyste.

Apports personnels

Au-delà des réalisations techniques, ce projet nous a permis d'acquérir et de consolider des compétences pluridisciplinaires rarement réunies dans un même projet académique :

— **Compétences SOC opérationnelles** : compréhension approfondie du cycle de vie d'un incident de sécurité, de la détection à la remédiation, en passant par le triage et l'escalade ;

— **Intégration système** : capacité à faire communiquer des outils hétérogènes (Wazuh, Shuffle, TheHive, Flask, APIs cloud) via des webhooks, des APIs REST et des formats JSON standardisés ;

— **Développement sécurisé** : conception d'APIs Python Flask, compilation d'exécutables C# (`quarantine.exe`, `isolate.exe`), gestion des tokens et des secrets en variables d'environnement ;

— **IA appliquée** : intégration de LLM dans un pipeline de décision critique, conception de prompts structurés, gestion des seuils de confiance et des formats de réponse JSON ;

— **Résolution de problèmes** : identification et résolution de cinq obstacles techniques majeurs (IP source Windows, conflits OpenSearch, DNS Docker Swarm, cache Active Response, compilation `csc.exe`).

Ce projet nous a également sensibilisés aux enjeux de la cybersécurité opérationnelle réelle : la difficulté du triage à grande échelle, l'importance de la traçabilité, et la nécessité d'un équilibre entre automatisation et jugement humain.

Perspectives d'évolution

Les travaux réalisés ouvrent plusieurs axes d'amélioration pour une évolution vers un déploiement de production :

1. **Passage à un LLM on-premise** avec Ollama pour éliminer la dépendance cloud et les risques de confidentialité liés à la transmission de données d'alerte à l'API Groq ;
2. **Couverture multi-agents dynamique** pour cibler automatiquement l'agent source de chaque alerte, rendant la plateforme scalable à un parc d'endpoints réel ;

3. **Extension des scénarios** aux vecteurs non couverts dans ce projet : mouvement latéral (MITRE T1021), exfiltration de données (T1048) et compromission de la chaîne d'approvisionnement ;
4. **Persistance des actions de réponse** via un pare-feu dédié (pfSense ou iptables) pour garantir que les blocages IP survivent aux redémarrages des endpoints ;
5. **Conformité réglementaire** : intégration d'un module de reporting automatique des incidents conforme aux exigences ISO/IEC 27035 et au référentiel NIS2.

En définitive, ce projet démontre qu'il est possible, avec des outils entièrement open source et un environnement de laboratoire accessible, de construire une plateforme SOC/SOAR fonctionnelle intégrant des capacités d'intelligence artificielle. Il constitue une base solide et reproductible pour tout projet visant à automatiser la réponse aux incidents dans un contexte réel.

Bibliographie

[1] National Institute of Standards and Technology, *Cybersecurity Framework 2.0*, https://www.nist.gov/cyberframework, Consulté le 10 juin 2026, 2024.

[2] SANS Institute, *Security Operations Center : Building, Operating, and Maintaining Your SOC*, https://www.sans.org/white-papers/, Consulté le 10 juin 2026, 2023.

[3] Gartner, *Market Guide for Security Orchestration, Automation and Response Solutions*, https://www.gartner.com/en/documents/security-soar, Consulté le 10 juin 2026, 2023.

[4] Shuffle Automation, *Shuffle — Open Source SOAR Documentation*, https://shuffler.io/docs, Consulté le 10 juin 2026, 2024.

[5] StrangeBee, *TheHive — Security Incident Response Platform*, https://docs.strangebee.com/thehive/, Consulté le 10 juin 2026, 2024.

[6] National Institute of Standards and Technology, *Guide to Computer Security Log Management — SP 800-92*, https://csrc.nist.gov/publications/detail/sp/800-92/final, Consulté le 10 juin 2026, 2006.

[7] Wazuh Inc., *Wazuh — Open Source Security Platform Documentation*, https://documentation.wazuh.com, Consulté le 10 juin 2026, 2024.

[8] MITRE Corporation, *MITRE ATLAS — Adversarial Threat Landscape for Artificial-Intelligence Systems*, https://atlas.mitre.org, Consulté le 10 juin 2026, 2024.

[9] Groq Inc., *Groq API Documentation — LLM Inference*, https://console.groq.com/docs, Consulté le 10 juin 2026, 2024.

[10] Meta AI, *Llama 3 — Large Language Model*, https://llama.meta.com, Consulté le 10 juin 2026, 2024.

[11] Pallets Projects, *Flask — Web Development, One Drop at a Time*, https://flask.palletsprojects.com, Consulté le 10 juin 2026, 2024.

[12] Oracle Corporation, *VirtualBox — User Manual*, https://www.virtualbox.org/manual/, Consulté le 10 juin 2026, 2024.

[13] Docker Inc., *Docker Swarm Mode Overview*, https://docs.docker.com/engine/swarm/, Consulté le 10 juin 2026, 2024.

[14] VirusTotal, *VirusTotal API v3 — Developer Reference*, https://developers.virustotal.com/reference, Consulté le 10 juin 2026, 2024.

[15] AlienVault, *Open Threat Exchange — OTX DirectConnect API*, https://otx.alienvault.com/api, Consulté le 10 juin 2026, 2024.

[16] MITRE Corporation, *MITRE ATT&CK Enterprise Matrix*, https://attack.mitre.org, Consulté le 10 juin 2026, 2024.